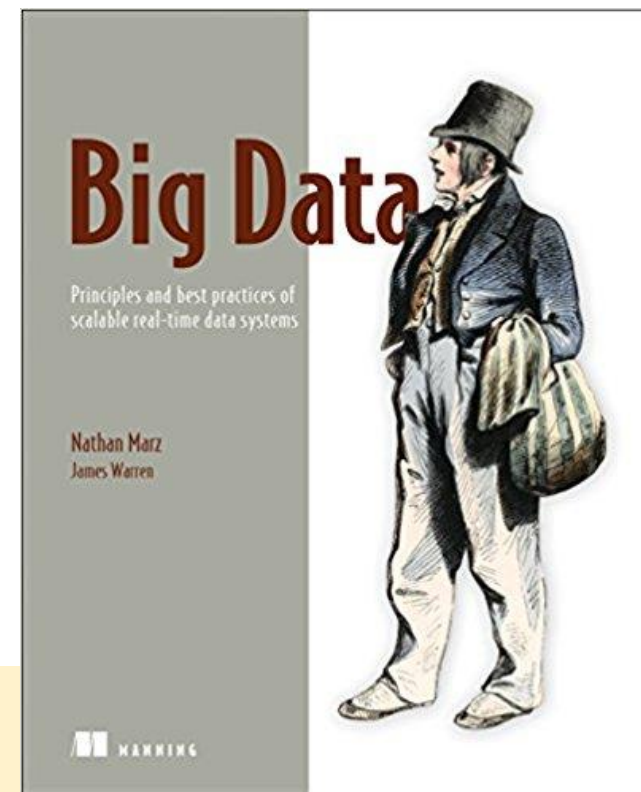




Big Data Architectures

Lambda Architecture



Limits of Traditional Tools



- Let's consider an example
 - Suppose your boss asks you to build a simple web analytics application. The application should track the number of pageviews to any URL a customer wishes to track. The customer's web page pings the application's web server with its URL everytime a pageview is received.
 - Additionally, the application should be able to tell you at any point what the top 100 URL's are by number of pageviews.
- ***Scaling issues with a traditional database.***

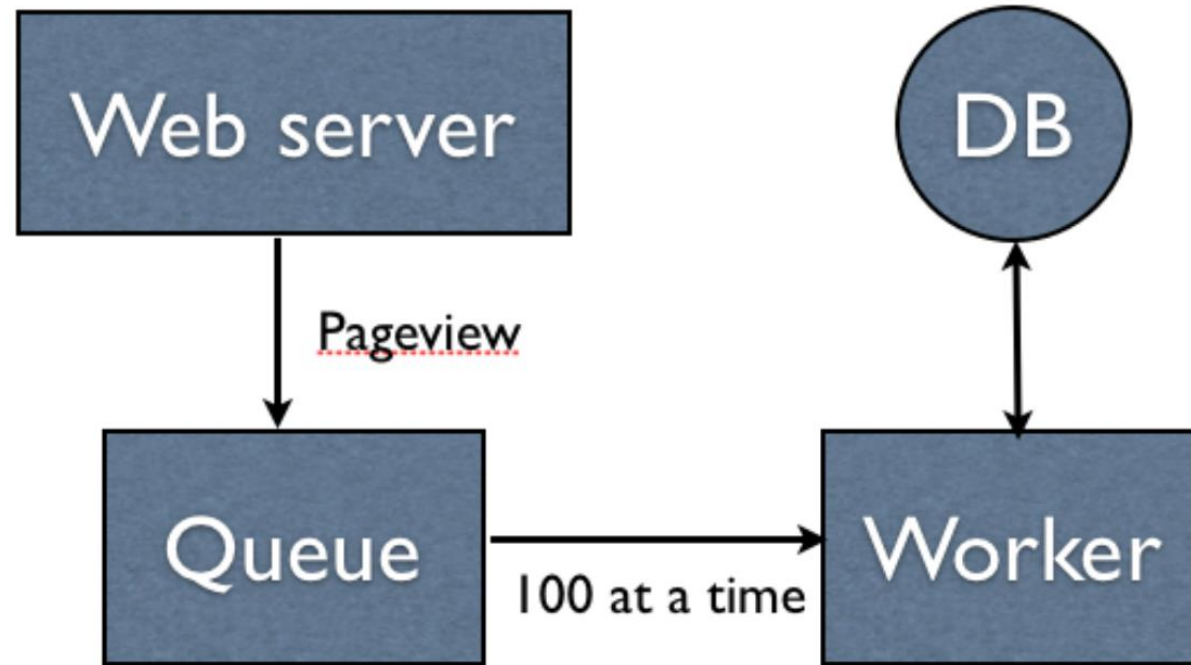
- The web analytics product is a huge success but you will incur very soon into the following problem

**TIMEOUT ERROR ON INSERTING
TO THE DATABASE!**

Limits of Traditional Tools



- **Scaling with a queue.** Resolves the timeout issues you were getting. It even has the added bonus that if the database ever gets overloaded again, the queue will just get bigger instead of timing out to the web server and potentially losing data.



Batching updates with queue and worker.

Limits of Traditional Tools



- ***Scaling by sharding the database.*** As the application gets more and more popular, you keep having to reshard the database into more shards to keep up with the write load. Each time gets more and more painful as there's so much more work to coordinate.

Limits of Traditional Tools



- ***Scaling by sharding the database.*** As the application gets more and more popular, you keep having to reshard the database into more shards to keep up with the write load. Each time gets more and more painful as there's so much more work to coordinate.
- ***Fault-tolerance issues begin.*** Eventually you have so many shards that it's not uncommon for the disk on one of the database machines to go bad. So that portion of the data is unavailable while that machine is down.

Limits of Traditional Tools



- ***Scaling by sharding the database.*** As the application gets more and more popular, you keep having to reshard the database into more shards to keep up with the write load. Each time gets more and more painful as there's so much more work to coordinate.
- ***Fault-tolerance issues begin.*** Eventually you have so many shards that it's not uncommon for the disk on one of the database machines to go bad. So that portion of the data is unavailable while that machine is down.
- ***Corruption issues.*** You don't notice until 24 hours later but by then the damage is done: many of the values in your database are inaccurate. Your system has no resilience to a human making a mistake.

Recap - Problems with traditional architecture



- *Fault-tolerance is hard*
- *Complexity pushed to application layer*
- *Lack of human fault-tolerance*
- *Maintenance is an enormous amount of work*

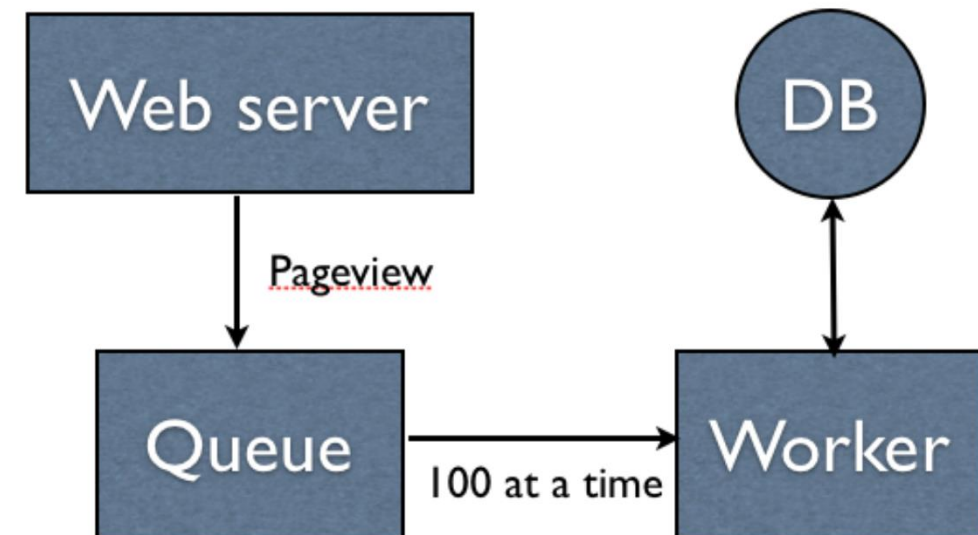
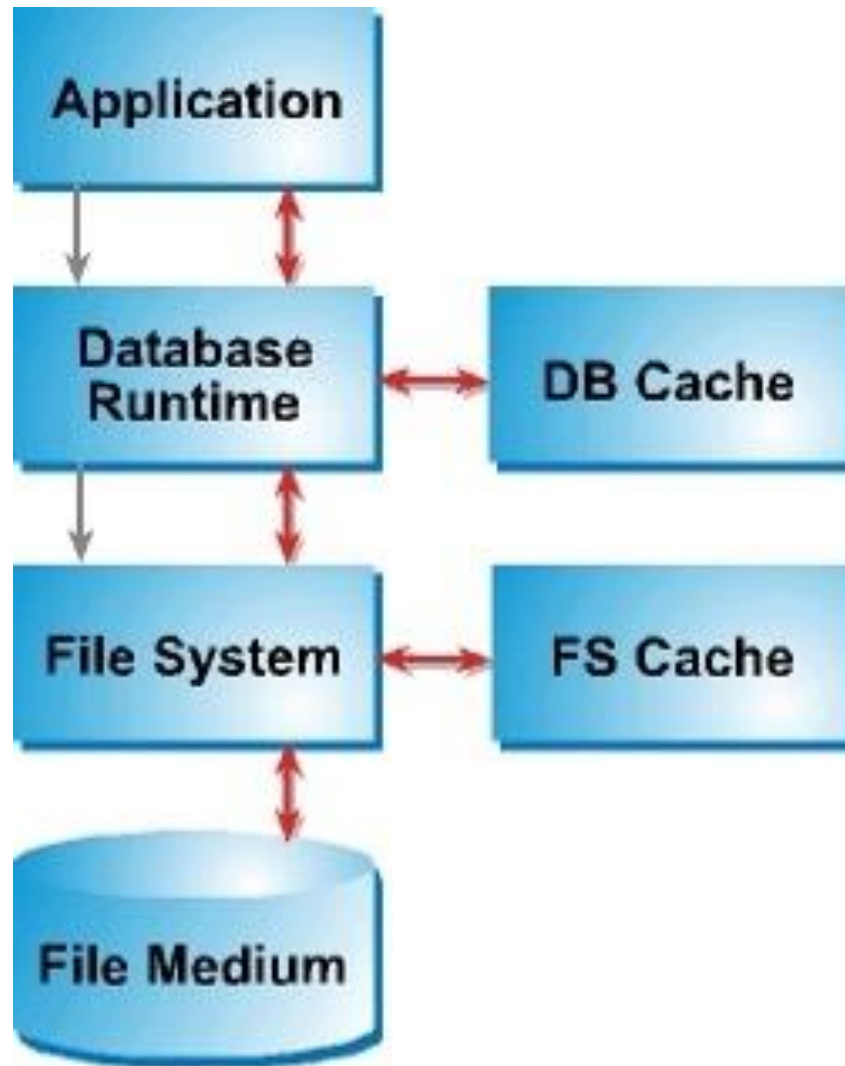
Recap - Problems with traditional architecture



- *Fault-tolerance is hard*
- *Complexity pushed to application layer*
- *Lack of human fault-tolerance*
- *Maintenance is an enormous amount of work*

**Solution: Big Data
Techniques & Technologies!!!**

Before Big Data Paradigm



Lambda Architecture



- ... Traditional systems have failed to scale Big Data
 - Not Scalable
 - Too Complex
- The paradigm shifted to **Lambda Architecture**

Speed layer

Serving layer

Batch layer

What is a informational system?



- A data system answers **questions based on information** that was **acquired in the past**

Big Data Techniques & Technologies



- The databases and computation systems you use for Big Data are **self-aware of their distributed nature**.
- Another core technique you will learn about is **making your data immutable**.
- Big Data techniques **can scale to so much data**, you have the ability to design systems in different ways.
- Hadoop, for example, can **parallelize large scale batch computations** on very large amounts of data, but the computations have high latency.
- NoSQL databases like Cassandra achieve their scalability by offering you a much **more limited data model** than you're used to with something like SQL.

Properties of a Big Data Information System



- ***Robust and fault-tolerant.*** Immutability and recomputation into the core of your systems to make your systems innately resilient to human error.
- ***Low latency reads and updates.*** The vast majority of applications require reads to be satisfied with very low latency, typically between a few milliseconds to a few hundred milliseconds.
- ***Scalable.*** Scalability is the ability to maintain performance in the face of increasing data and/or load by adding resources to the system.
- ***General.*** Lambda Architecture generalizes to applications as diverse as financial management systems, social media analytics, scientific applications, and social networking.
- ***Extensible.*** Extensible systems allow functionality to be added with a minimal development cost.
- ***Minimal maintenance***
- ***Debuggable***

Lambda Architecture



Batch Layer

- Stores the immutable constantly growing dataset
- Computes arbitrary views from this dataset using BigData technologies (can take hours)
- Can be always recreated

Serving Layer

- Indexing and exposing
 - the pre-computed batch views
 - the incremented real-time views
- Merges the batch and the real-time views

Speed Layer

- Computes views from the data stream in real-time
- Compensate for the high latency of the batch layer
- Incremental model and views are transient

Batch vs. Stream Processing



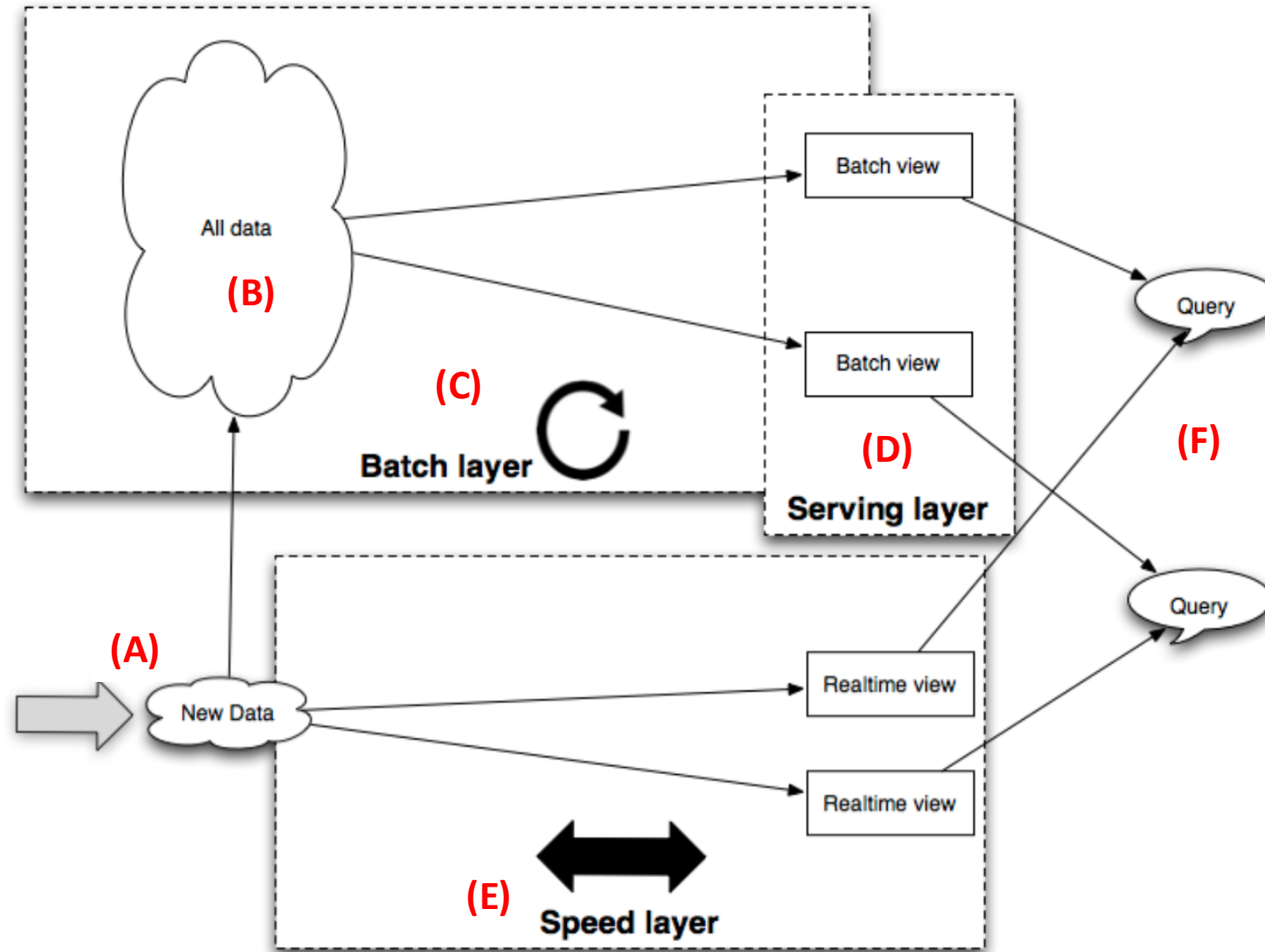
- Batch processing

- Gathering of data and processing as a group at one time.
- Jobs run to completion
- Data might be out of date

- Real-time processing

- Processing of data that takes place as the information is being entered.
- Run for ever

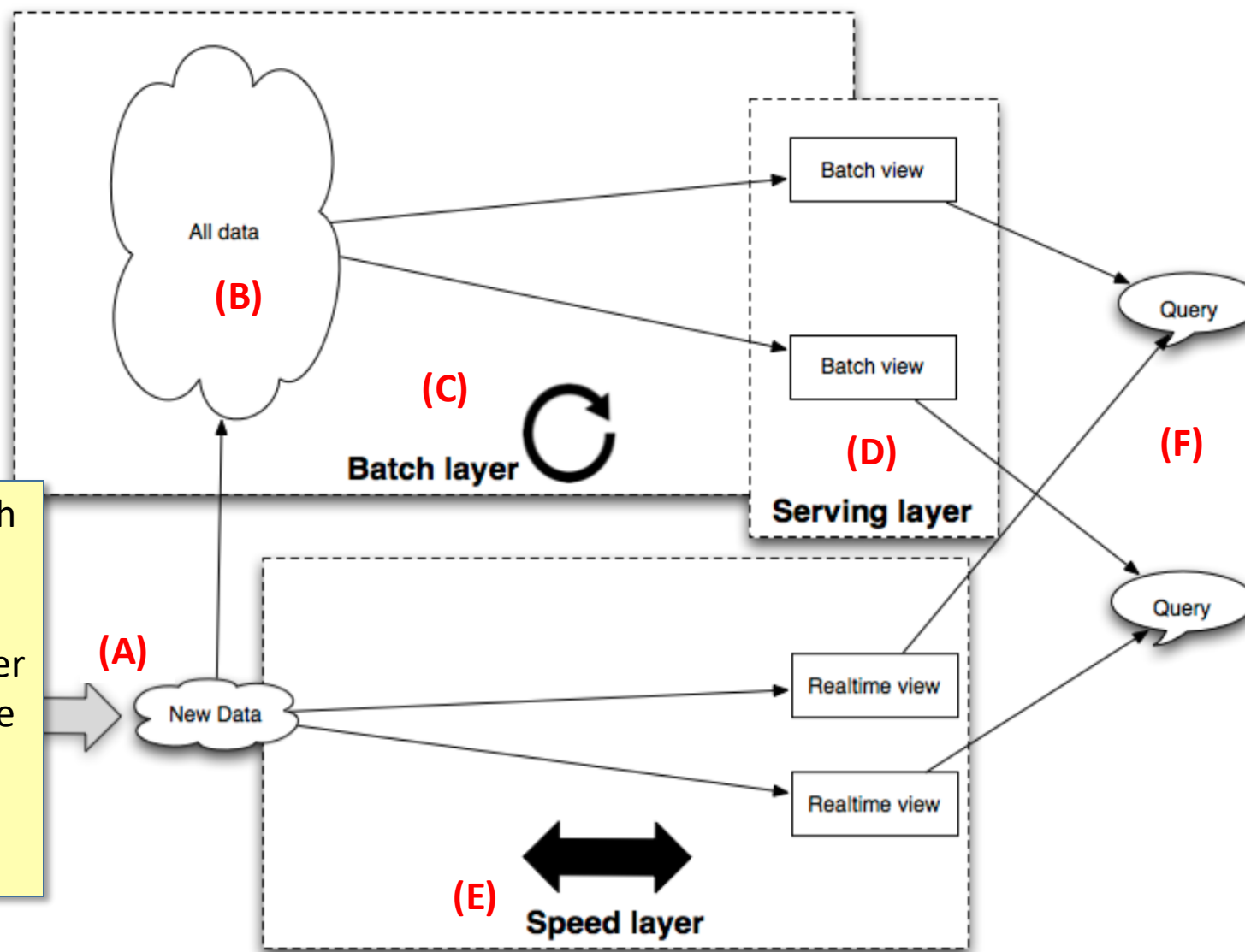
Lambda Architecture



Lambda Architecture



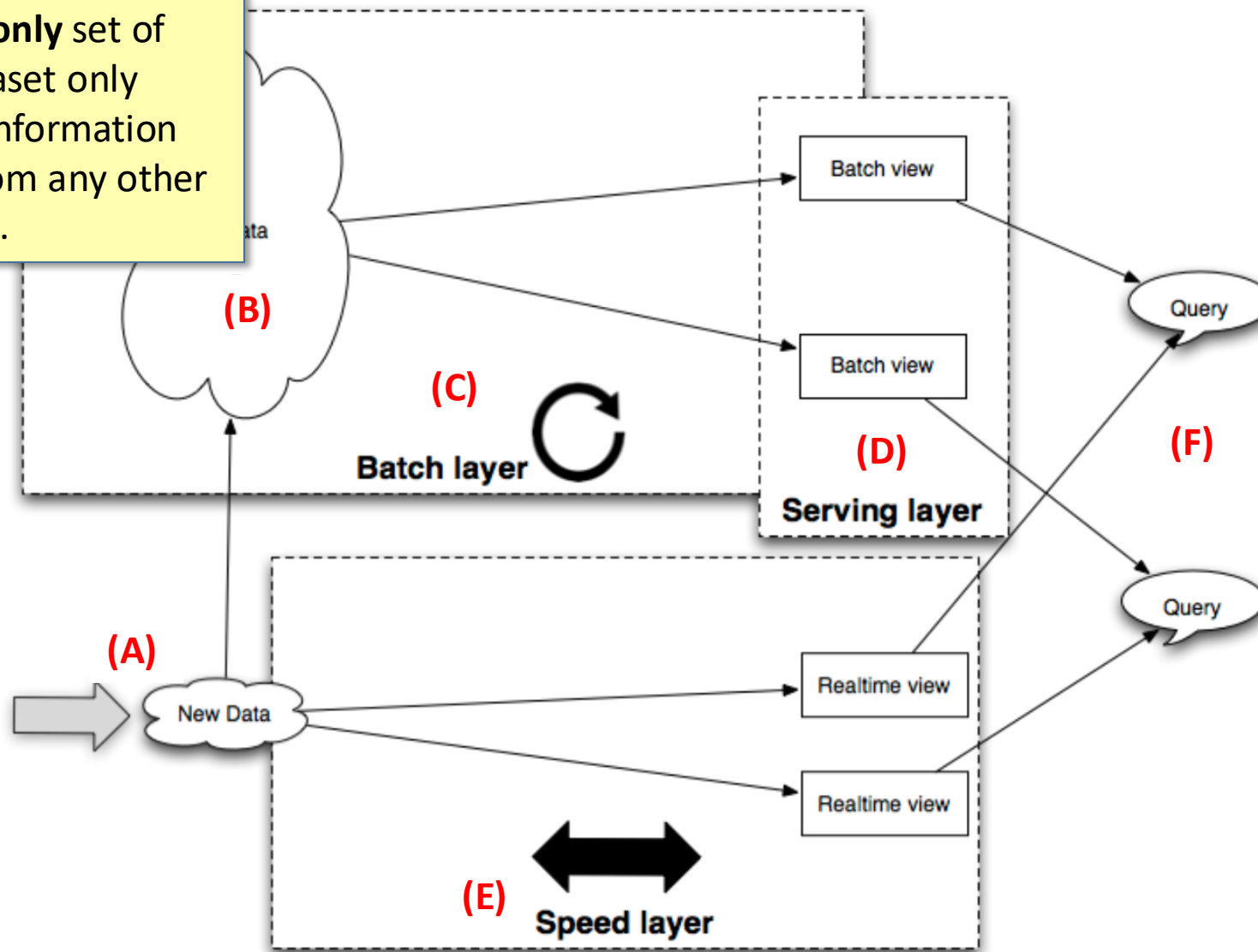
(A) All new data is sent to both the batch layer and the speed layer. In the batch layer, new data is appended to the master dataset. In the speed layer, the new data is consumed to do incremental updates of the realtime views.



Lambda Architecture



(B) The master dataset is an **immutable, append-only** set of data. The master dataset only contains the **rawest** information that is not derived from any other information you have.



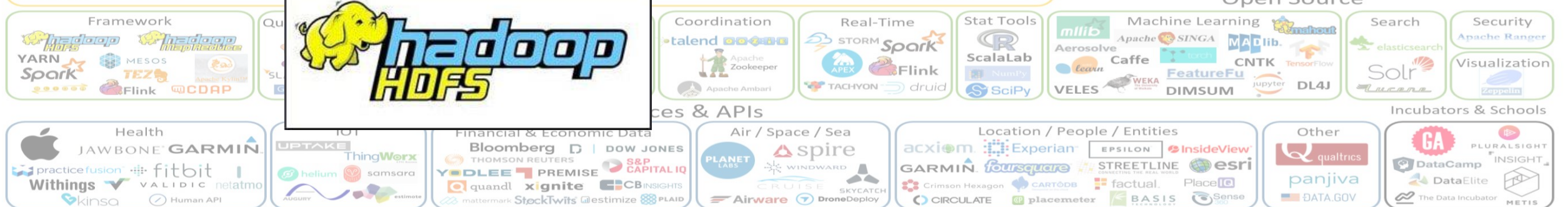
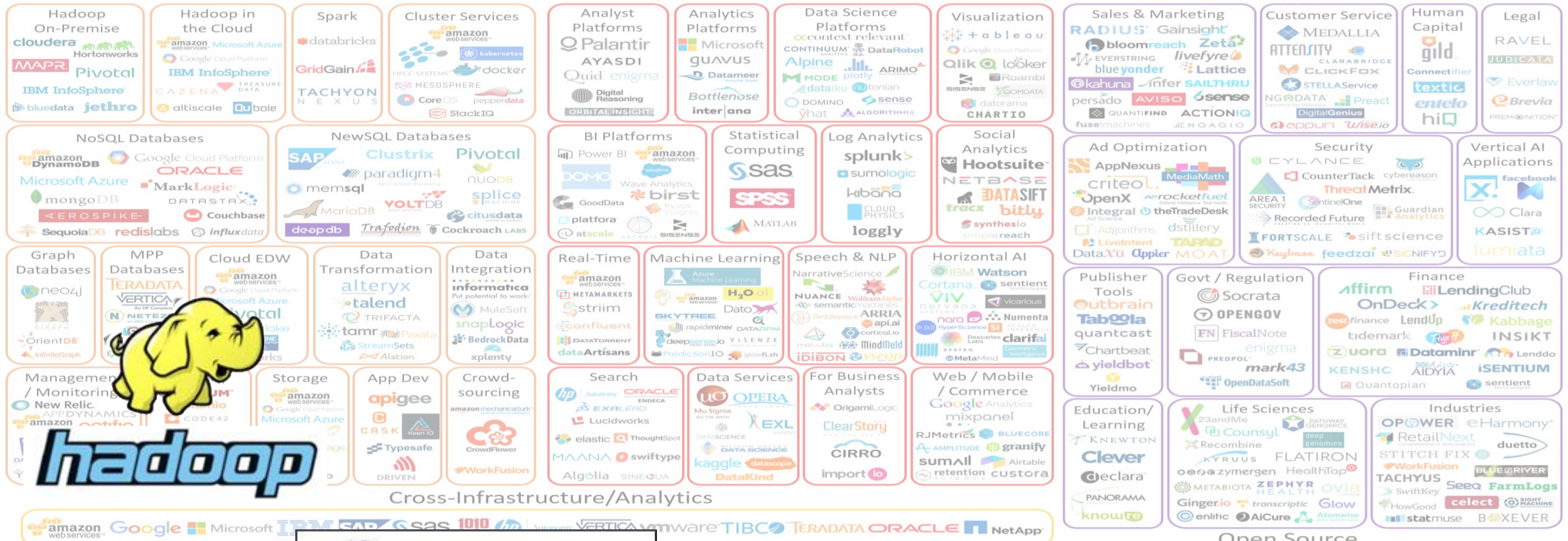
Big Data Processing Technologies



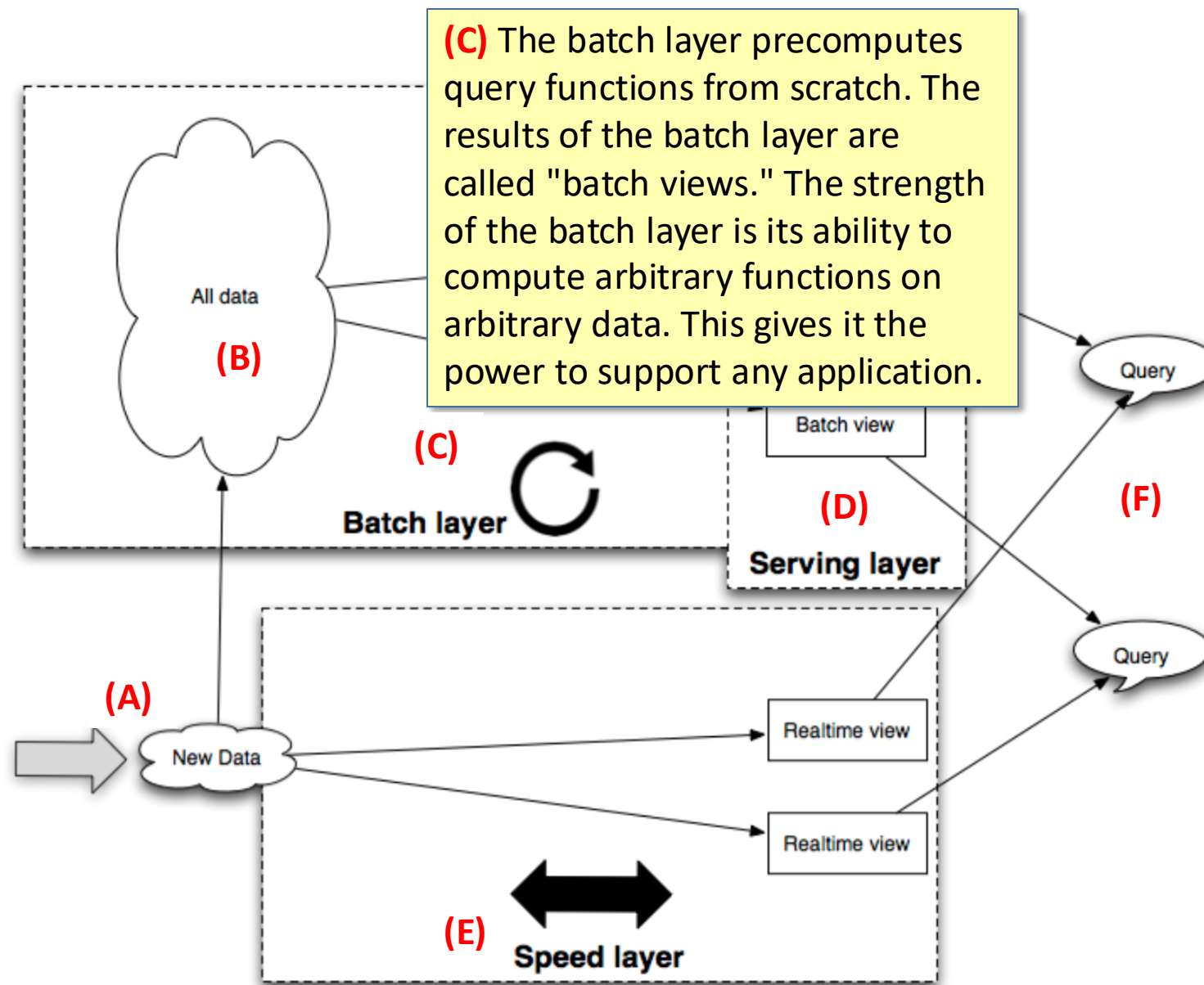
Infrastructure

Analytics

Applications



Lambda Architecture



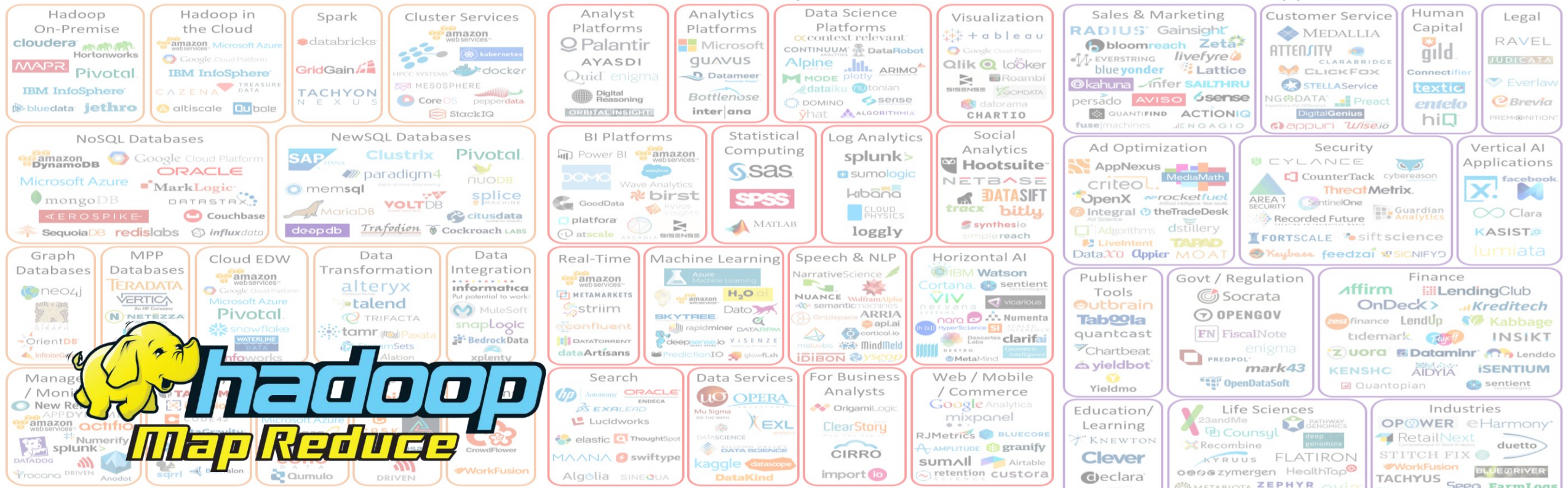
Big Data Processing Technologies



Infrastructure

Analytics

Applications



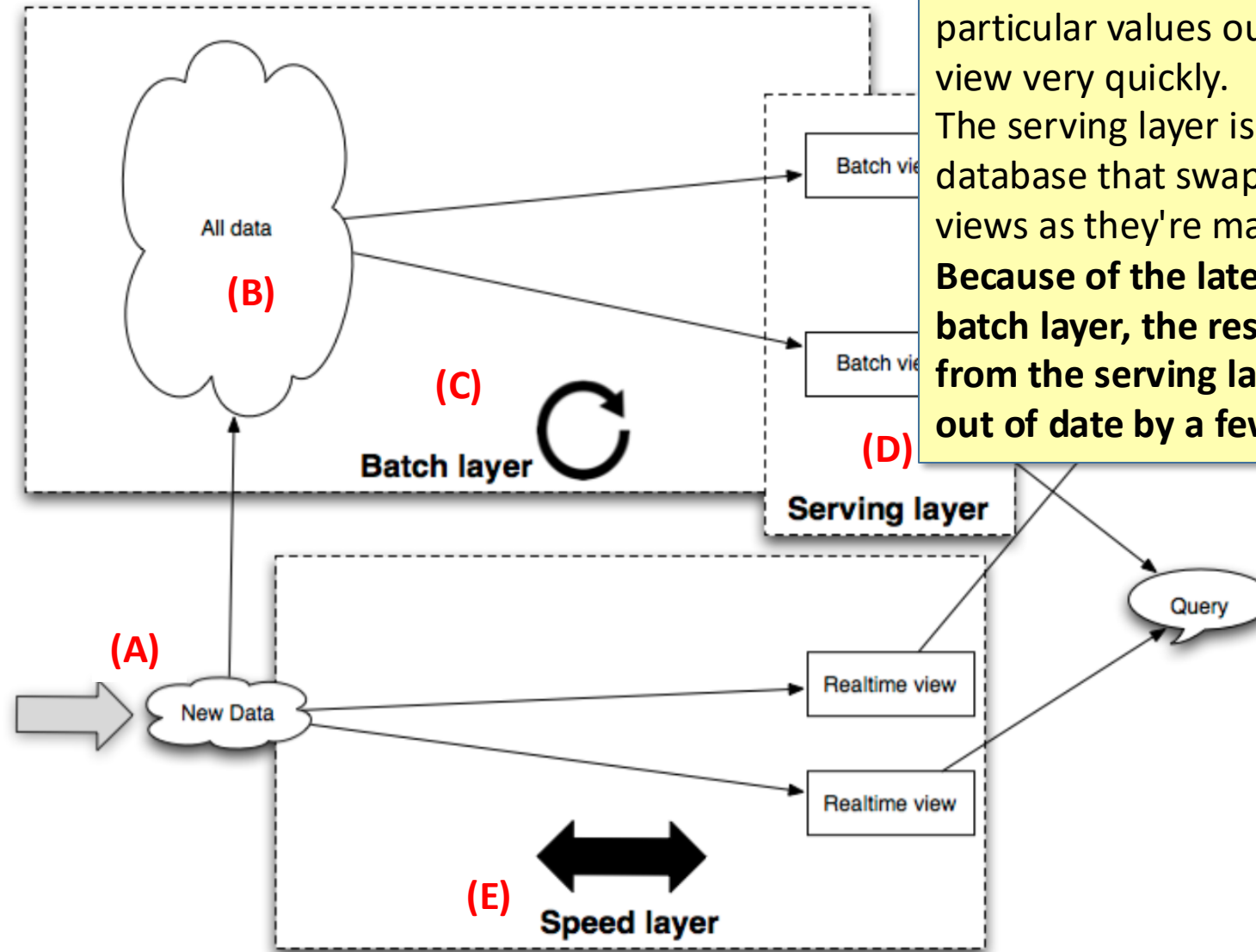
Cross-Infrastructure/Analytics



Data Sources & APIs



Lambda Architecture



(D) The serving layer indexes the batch views produced by the batch layer and makes it possible to get particular values out of a batch view very quickly. The serving layer is a scalable database that swaps in new batch views as they're made available. **Because of the latency of the batch layer, the results available from the serving layer are always out of date by a few hours.**

Big Data Storage Technologies



Infrastructure

Analytics

Applications

Infrastructure

- Hadoop On-Premise:** cloudera, Hortonworks, MAPR, Pivotal, IBM InfoSphere, bluedata, jethro
- Hadoop in the Cloud:** amazon web services, Microsoft Azure, Google Cloud Platform, IBM InfoSphere, CAZENA, TREASURE DATA, altiscale, Quale
- Spark:** databricks, GridGain, TACHYON, NEXUS
- Cluster Services:** amazon web services, kubernetes, docker, MESOSPHERE, Core OS, pepperdata, StackIQ
- Analyst Platforms:** Palantir, AYASDI, Quid, enigma, Digital Reasoning, ORBITAL INSIGHT
- Analytics Platforms:** Microsoft, guavus, Datameer, Bottlenose, interlana
- Data Science Platforms:** context relevant, DataRobot, Alpine, MODE, plotly, ARIMO, dataiku, tonian, DOMINO, sense, yhat, ALGORITHMIA
- Visualization:** tableau, Google Cloud Platform, Qlik, looker, Roambi, SISense, COMDATA, datorama, CHARTIO
- Sales & Marketing:** RADIUS, Gainsight, bloomreach, Zeta, EVERSTRING, livefyre, blueyonder, Lattice, kahuna, infer, SAILTHRU, persado, AVISO, sense, QUANTIFIND, ACTIONIQ, fuse|machines, JENGAGIO
- Customer Service:** MEDALLIA, ATTENITY, CLARABRIDGE, CLICKFOX, STELLA Service, NGDATA, Preact, DigitalGenius, appur, Wiseio
- Human Capital:** gild, Connectifier, textio, entelo, hiQ
- Legal:** RAVEL, JUDICATA, Everlaw, Brevia, PREMONITION
- NoSQL Databases:** amazon DynamoDB, Google Cloud Platform, Microsoft Azure, mongoDB, MarkLogic, DATASTAX, Couchbase, SEQUOIA DB, redislabs, influxdata
- NewSQL Databases:** SAP HANA, Clustrix, Pivotal, paradigm4, nuodb, splice, memsql, MariaDB, VOLTDB, citusdata, deepdb, Trafodion, Cockroach LABS
- BI Platforms:** Power BI, amazon web services, Domo, Wave Analytics, birst, GoodData, kyvos, insights, platforma, atscale, ARCADIA, SISense
- Statistical Computing:** sas, SPSS, MATLAB
- Log Analytics:** splunk, sumologic, kibana, CLOUD PHYSICS, loggly
- Social Analytics:** Hootsuite, NETBASE, DATASIFT, tracx, bitly, synthesio, simple reach
- Ad Optimization:** AppNexus, MediaMath, critico, OpenX, rocketfuel, Integral, theTradeDesk, Ad Science, Ad Algorithms, distillery, LiveIntent, TAPAD, Moat
- Security:** CYLANCE, CounterTack, cybereason, AREA 1 SECURITY, SentinelOne, Recorded Future, Guardian Analytics, FORTSCALE, sift science, Keybase, feedzai, SIGNIFYD
- Vertical AI Applications:** facebook, Clara, KASIST, lumiata
- Graph Databases:** neo4j, STAPH, OrientDB, InfiniteGraph
- MPP Databases:** TERADATA, VERTICA, AN HP COMPANY, Netezza, Qlik, Action, cognitio, XSQL, dremio
- Cloud EDW:** amazon web services, Google Cloud Platform, Microsoft Azure, Pivotal, snowflake, WATERLINE DATA, Infoworks
- Data Transformation:** alteryx, talend, TRIFACTA, tamr, Paxata, StreamSets, Alation
- Data Integration:** informatica, Put potential to work, MuleSoft, snapLogic, Bedrock Data, xplenty
- Real-Time:** amazon web services, METAMARKETS, striim, confluent, DATATORRENT, dataArtisans
- Machine Learning:** Azure Machine Learning, amazon web services, H2O, SKYTREE, Dato, rapidminer, DATAIPMA, deepzone, VIZEN, PredictionIO, glowfish
- Speech & NLP:** NarrativeScience, NUANCE, WolframAlpha, semantic machines, ARRIA, api.ai, Gridspace, cortical.io, maluba, MindMeld, iBIBON, vseeop
- Horizontal AI:** IBM Watson, Cortana, sentient, viv, nervana, vicarious, nana, Numenta, HyperScience, SI, clarifai, DEXTER, MetaMind
- Publisher Tools:** Outbrain, Taboola, quantcast, Chartbeat, yieldbot, Yieldmo
- Govt / Regulation:** Socrata, OPENGOV, FN, FiscalNote, enigma, mark43, PREDPOL, OpenDataSoft
- Finance:** Affirm, LendingClub, OnDeck, Kreditech, LendUp, Kabbage, tidemark, Ruff, ZUORA, Dataminr, Lenddo, KENSHC, iSENTIUM, QUANT
- Management / Monitoring:** New Relic, APPDYNAMICS, amazon web services, octifio, Numerify, splunk, DATADOG, TROCAN, DRIVEN, Anodot
- Security:** TANIUM, Illumio, CODE42, DataGravity, CipherCloud, VECTRA, sqrrl, BlueTalon
- Storage:** amazon web services, Google Cloud Platform, Microsoft Azure, panasas, nimblestorage, COHO, Qumulo
- App Dev:** apigee, CASK, Keen IO, Typesafe, DRIVEN
- Crowd-sourcing:** amazon mechanical turk, CrowdFlower, WorkFusion
- Search:** hp, Autonomy, ORACLE, ENDECA, EXALEAD, Lucidworks, elastic, ThoughtSpot, MAANA, swifttype, Algolia, SINEQUA
- Data Services:** UC OPERA, Mu Sigma, EXL, DATA SCIENCE, kaggle, dataSCOPE, DataKind
- For Business Analysts:** OrigamiLogic, ClearStory, CIRRO, import io
- Web / Mobile / Commerce:** Google Analytics, mixpanel, RJMetric, CORE, AMP, SUMA, rete

Cross-Infrastructure/Analytics

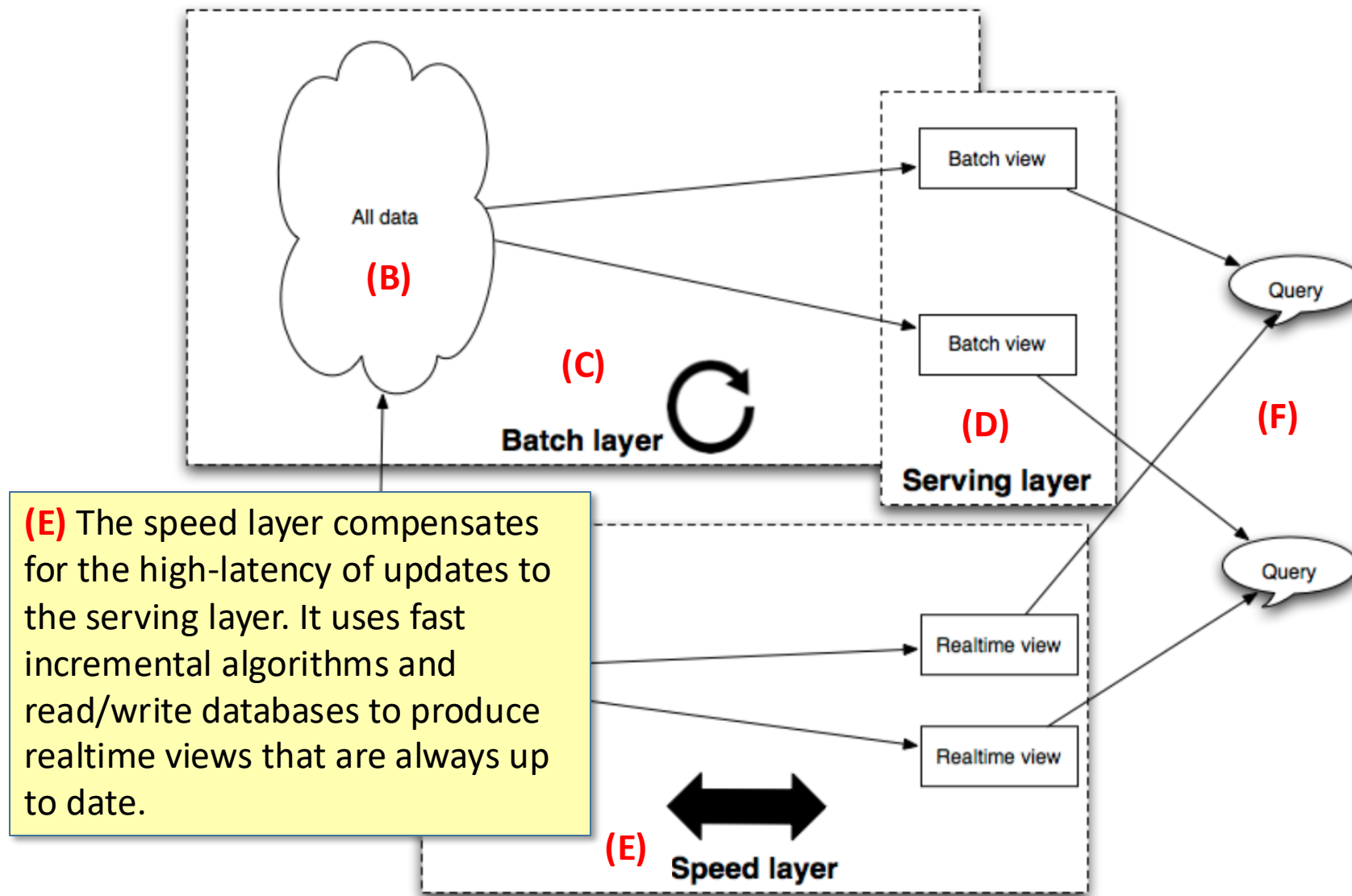
amazon web services, Google, Microsoft, IBM, SAP, SAS, data, hp, Autonomy, VERTICA, An HP Company, vmware, TIBCO, TERADATA, ORACLE, Facebook, SAP



Open Source

- Framework:** hadoop, YARN, Spark, MESOS, TEZ, Flink, CDAP
- Query / Data Flow:** SLAMDATA, HIVE, DRILL, Google Cloud Dataflow
- Data Access:** cassandra, HBASE, mongoDB, kafka, CouchDB, riak, OPENTSDB, nifi
- Coordination:** talend, Apache Zookeeper, Apache Ambari
- Real-Time:** STORM, Spark, APEX, Flink, TACHYON, druid
- Stat Tools:** ScalaLab, NumPy, SciPy
- Machine Learning:** milib, Aerosolve, Caffe, WEKA, VELES, DIMSUM
- Security:** Apache Ranger, Apache Zeppelin
- Health:** JAWBONE, GARMIN, practicefusion, fitbit, Withings, VALIDIC, netatmo, kinsa, Human API
- IOT:** UPTAKE, ThingWorx, helium, samsara, augury, estimate
- Financial & Economic Data:** Bloomberg, DOW JONES, THOMSON REUTERS, YODLEE, PREMISE, S&P CAPITAL IQ, quandl, xignite, CBINSIGHTS, mattermark, StockTwits, estimate, PLAID
- Air / Space / Sea:** PLANET LABS, spire, WINDWARD, CRUISE, SKYWATCH, Airware, DroneDeploy
- Location / People / Entities:** axiomatic, Experian, EPSILON, InsideView, GARMIN, foursquare, STREETLINE, esri, Crimson Hexagon, CARTO DB, factual, PlaceIQ, CIRCULATE, placemeter, BASIS, Sense
- CouchDB relax**
- Others:** panjiva, DATA.GOV, The Data Incubator, METIS

Lambda Architecture



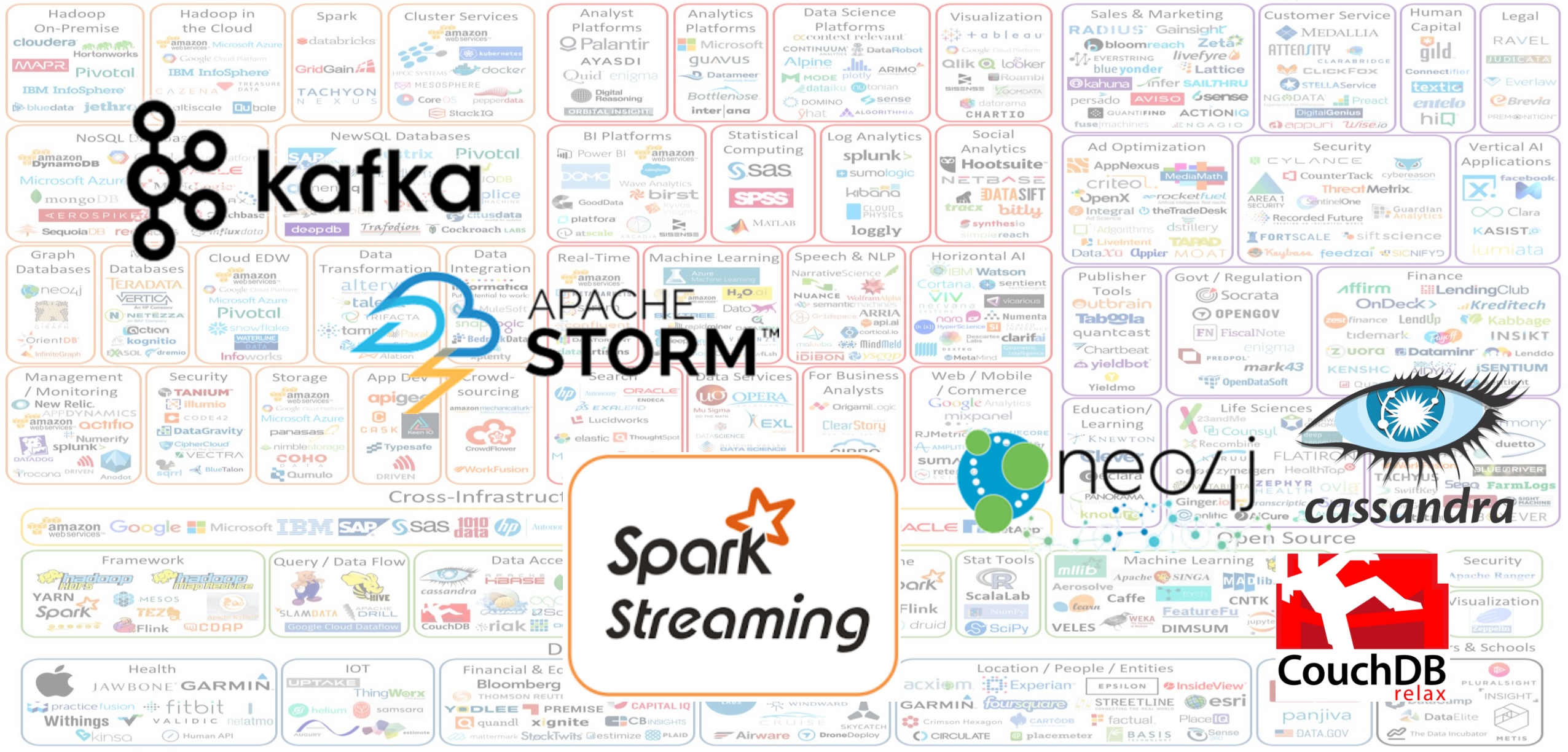


Big Data Processing Technologies

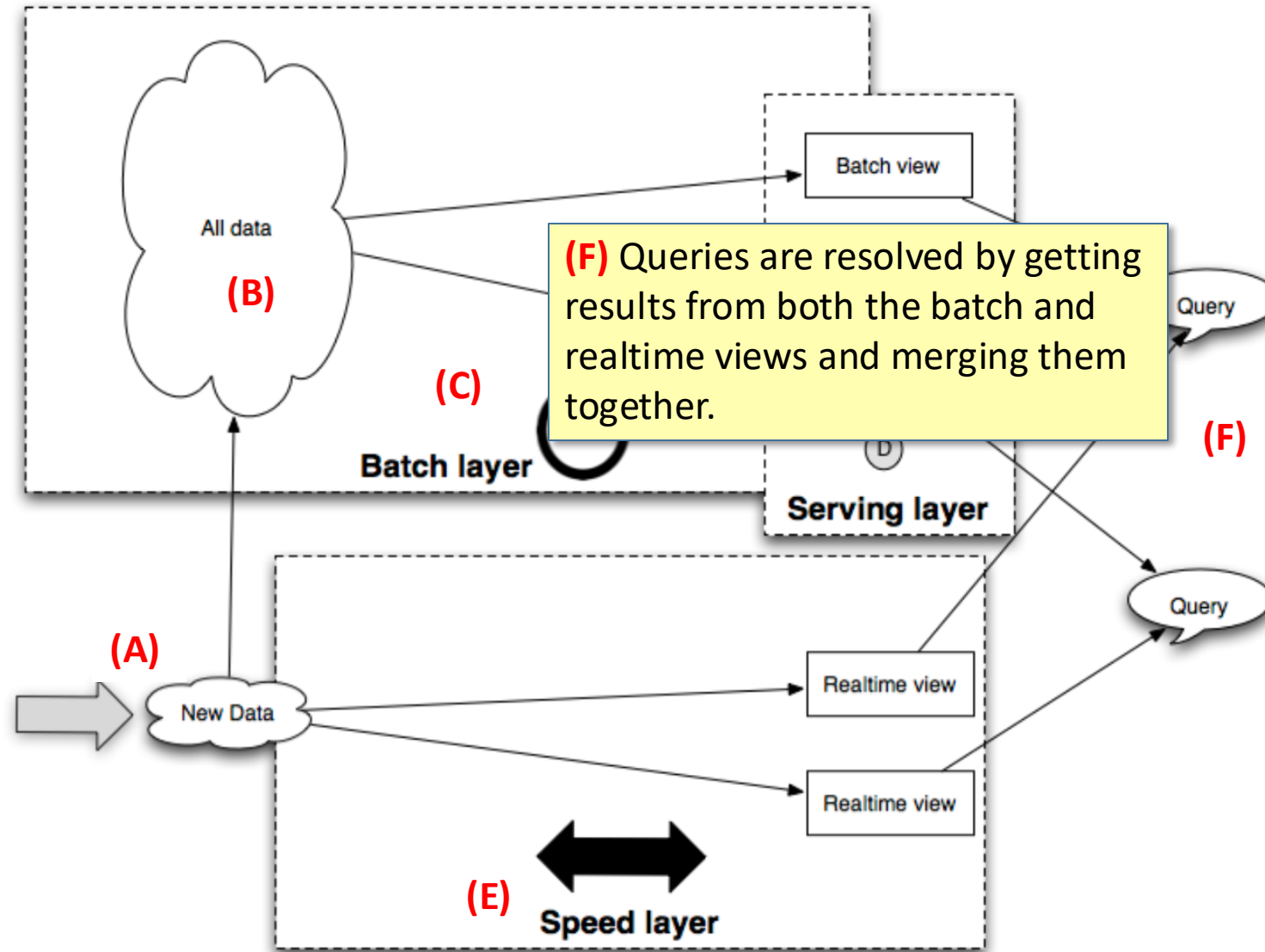
Infrastructure

Analytics

Applications



Lambda Architecture



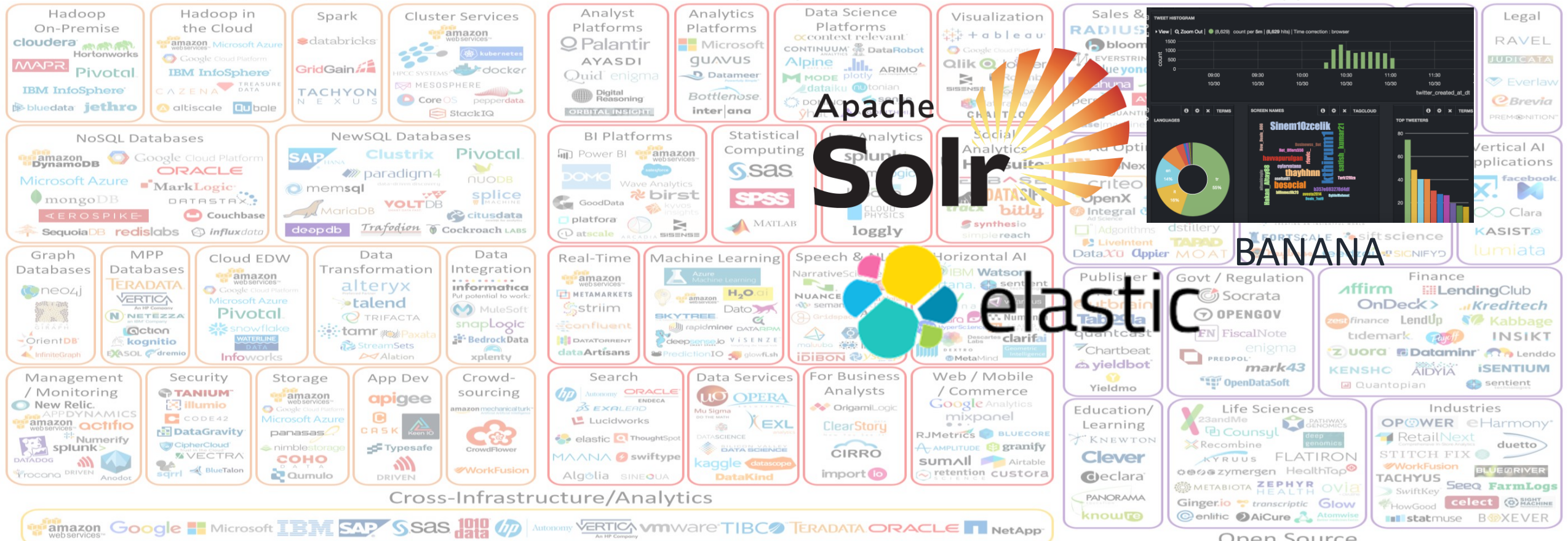
Big Data Storage/Searching Technologies



Infrastructure

Analytics

Applications

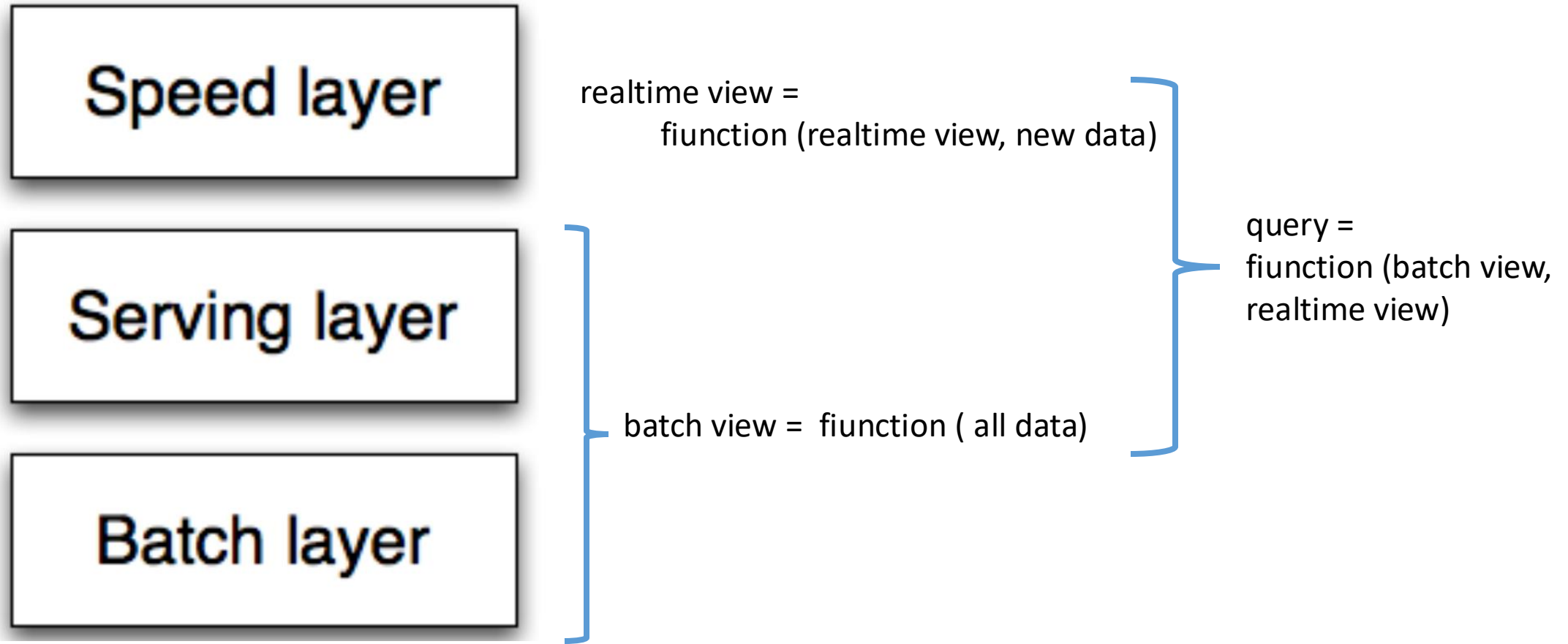


Apache
Solr
Elastic



BANANA

Lambda Architecture



Traditional Approach



- Enterprise systems generally have one or more large relational databases at their center. These databases hold “the current state” of the system, and other applications query and update this state. However this approach has problems with race-conditions and performance bottlenecks.
- Sometimes such systems keep “history” information (aka audit trails) which theoretically show how the current state has been reached.



- For the reflection: Questioning the Lambda Architecture

<https://www.oreilly.com/ideas/questioning-the-lambda-architecture>

<http://milinda.pathirage.org/kappa-architecture.com/>



Project Works

"Requirements"



- **Data Ingestion:**

- Apache NiFi, Streaming API, MQTT, ...
- Public dataset file (for group of 2 people)

- **Data Storage**

- HDFS, Big Data File Formats, NoSQL DB, ...

- **Data Processing**

- Batch and Interactive Query:
 - ❖ Apache Spark, Pig, HIVE, ...
- Real-time Analytics: at least one query (for group > 2 students)
 - ❖ Spark Streaming, Storm, KSQL, ...

- **Data Visualization**

- Apache Solr + BANANA, Elasticsearch + KIBANA, Zeppelin, D3.js, React, ...

How to collect data



- Datasets
- Endpoints
- API
- Scraping
- MQTT
- ...

Some Ideas



- IoT Analytics: sensors, smartphone, raspberry router, etc.
- Social Media: Twitter Streaming API, ...
- Web Analytics: By Scraping Booking.com, Subito.it, ...
- Epidemiological and Genomics Analytics: COVID19 dataset are available
- Cyber Intelligence: Open Source Intelligence
- ...

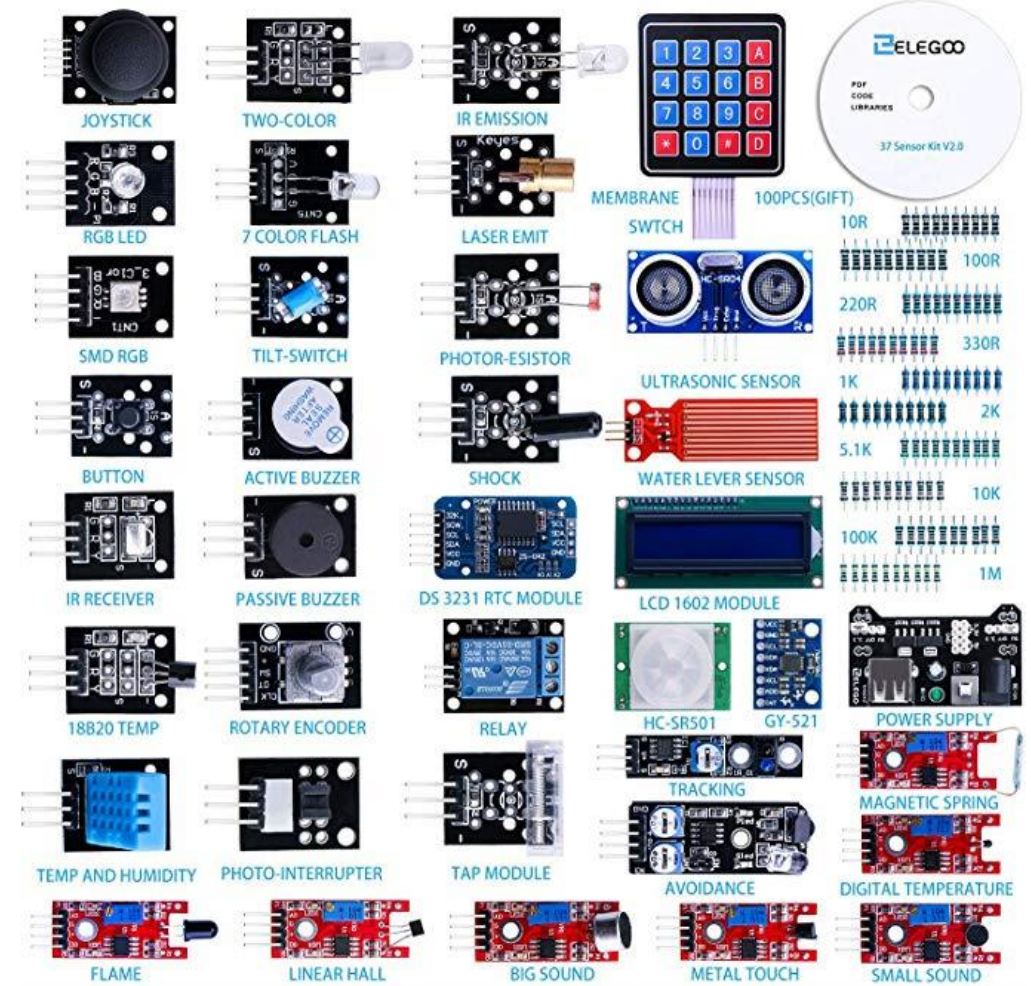
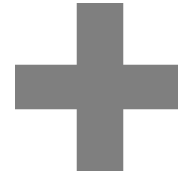
Smartphone Sensors



- Let's try to collect data from the smartphone, in order to perform some analytics about my activities and show them into an interactive dashboard
- **Data Ingestion:** <https://whatwebcando.today/device-position.html>
- **Data Storage:** choose and use the data file format, choose the repository (HDFS, KeyValue store, and so forth)
- **Data Processing:** let us assign some analytics about my activities, pre-compute and then show the results in a live dashboard, define stream or batch processing that you need
- **Data Visualization:** use D3.js, ApacheSolr + BANANA, Elasticsearch + KIBANA, or whatever you want to visualize the results obtained by analyzing your data log
- **Note:** *you may use thirdparty code (github, etc.) for implementing the activity recognition is not important complexity or accuracy of the task*



Temperature and Humidity in the classroom



Classroom Temperature and Humidity



- Let's try to use collect data from router for implementing things like, parental control or simply a network traffic analytics of my LAN and show them into n interactive dashboard
- **Data Ingestion:** collect data by using Arduino and MQTT + Mosquitto Broker
- **Data Storage:** choose and use the data file format, choose the repository (HDFS, KeyValue store, and so forth)
- **Data Processing:** Percentage of humidity in the environment; Heat Index is calculated; etc.
- **Data Visualization:** use D3.js, ApacheSolr + BANANA, Elasticsearch + KIBANA, or whatever you want to visualize the results obtained by analyzing your data log
- **Note:** I will give you some ESP32 and DHT11 sensors, or other sensors among the available ones <https://create.arduino.cc/projecthub/Gersaibot/temperature-streaming-with-arduino-big-data-tools-eb22fc>.



Raspberry Router



- Let's try to use collect data from router for implementing things like, parental control or simply a network traffic analytics of my LAN and show them into an interactive dashboard
- **Data Ingestion:** collect data about IPs, content topics, etc., considering the time dimension
- **Data Storage:** choose and use the data file format, choose the repository (HDFS, KeyValue store, and so forth)
- **Data Processing:** analytics question about network activities, pre-compute the results and then show them in a live dashboard, define stream or batch processing that you need
- **Data Visualization:** use D3.js, ApacheSolr + BANANA, Elasticsearch + KIBANA, or whatever you want to visualize the results obtained by analyzing your data log
- **Note:** I will give you raspberry, if needed, then you can collect data from it by configuring smart gateway following this guide <http://rbnet.it/guide/parental-control-con-raspberry-pi-squid-squidguard-e-dnsmasq/>



- Let's collect data from web or social media, and try to implement some interesting service to discover relevant information (OSIF)
- **Data Ingestion:** identify sources of information and what kind of resources you would like to manage, and for which objectives?
- **Data Storage:** choose and use the data file format, choose the repository (HDFS, KeyValue store, and so forth)
- **Data Processing:** use existing algorithms for pre-computing the results and then show them in a live dashboard.
- **Data Visualization:** use D3.js, ApacheSolr + BANANA, Elasticsearch + KIBANA, or whatever you want to visualize the results obtained by analyzing your data log
- **Note:** for instance let's find similar user profiles, or let's geolocate a selfie posted on social media, let's find if some selected users have same connection time on some network.

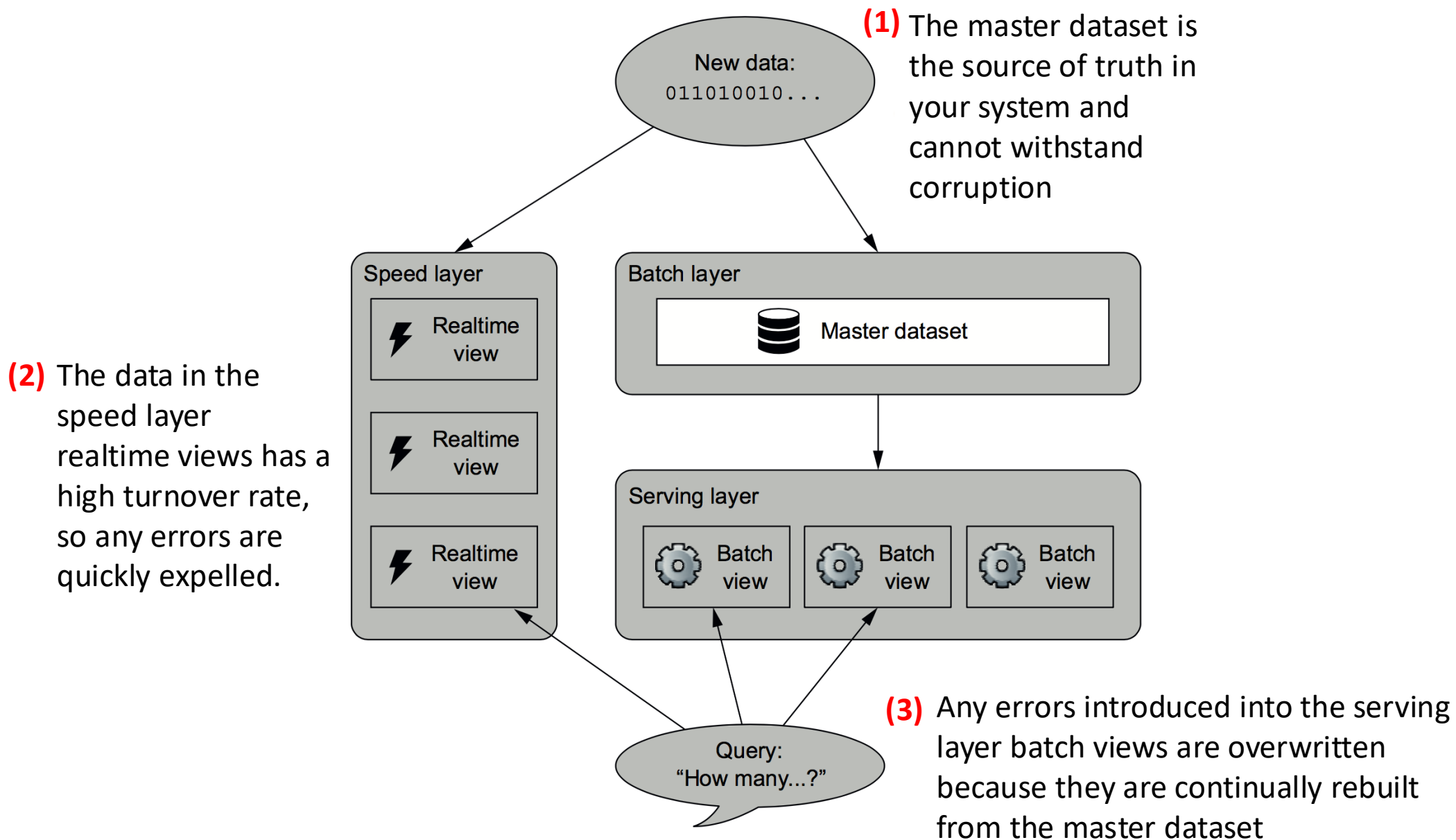
- Suggested video:[ITA] <https://www.youtube.com/watch?v=kYdP0BIGQAM>
- Visit: <https://www.docker.com/get-started>
- Download: Docker Desktop
- Let's start with docker next week for using HDFS and MapReduce
 - <https://github.com/effereds/hadoop>
 - From your terminal: `docker pull efferre/hadoop`



2nd Lesson – BATCH LAYER

Master Data Storage

Data Workflow



Master Data



Data Relevance Tradeoff



Data is
just too valuable
to delete!
**We must
store anything!**

It depends ...

Big Data technologies allow **to store the raw information** from new and existing data sources so that you can **later use it** to create **new data-driven products**, which you haven't **thought about today!**

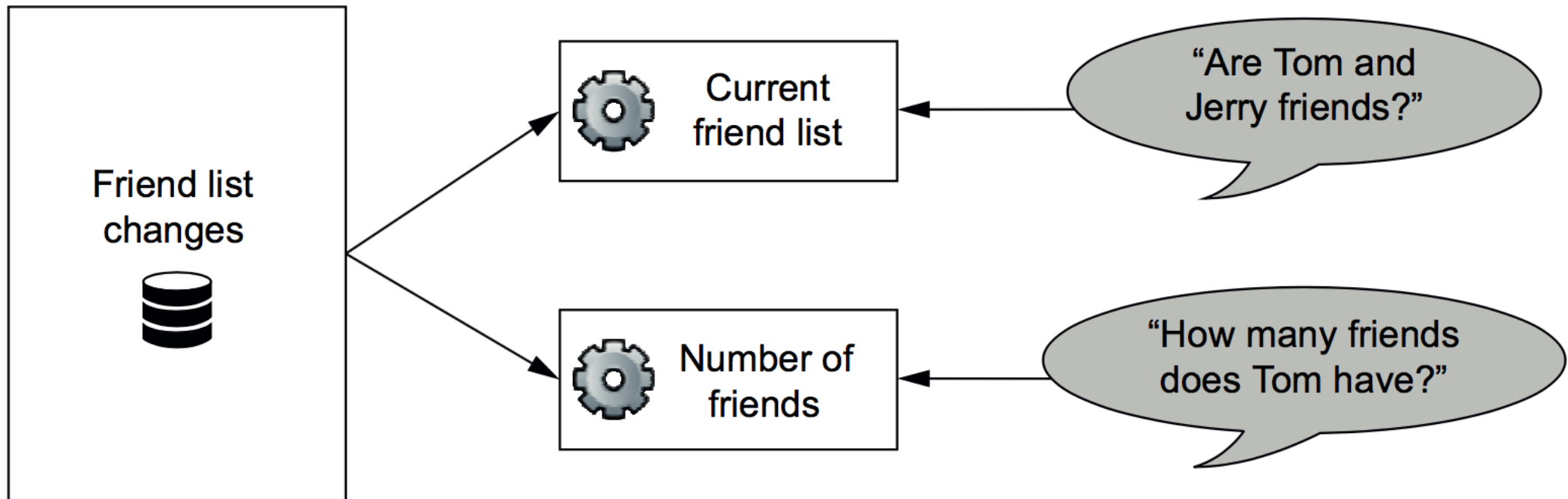


Nonsense! Just
store the data
**you know
you need today!**



- **Information** is the general collection of knowledge relevant to your Big Data system.
- **Data** refers to the information that **can't be derived** from anything else. Data serves as the axioms from which everything else derives.
- **Queries** are questions you ask to your data.
- **Views** are information that has been **derived from your data**.

Data/Query/Views

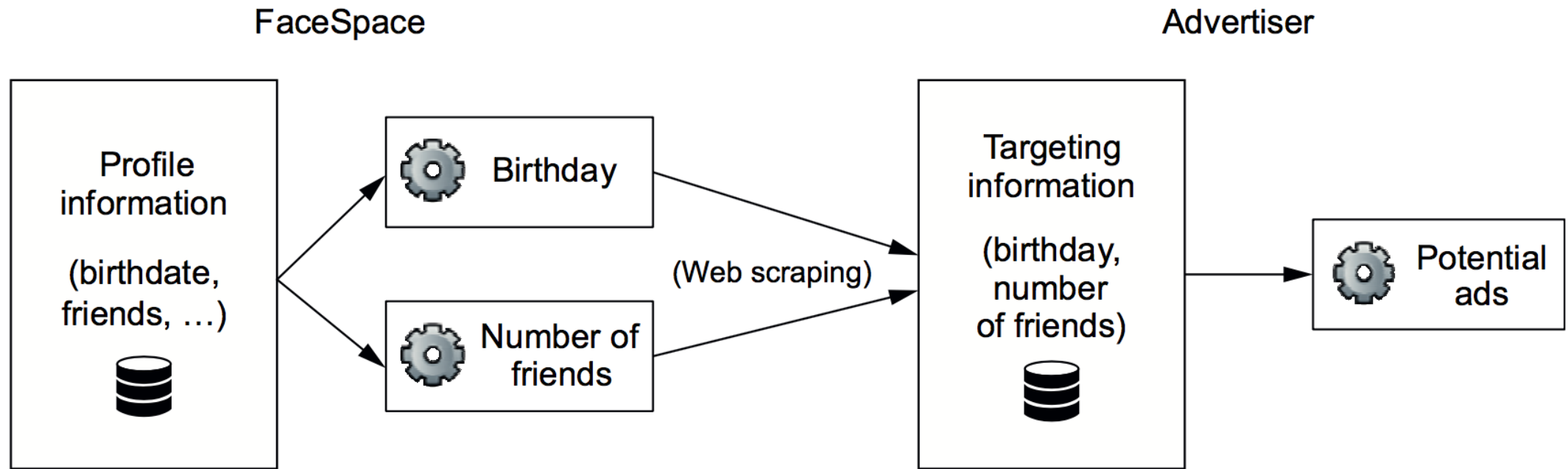


1 Your data is information that cannot be derived from anything else.

2 The views are computed from the data to help answer queries.

3 The queries you want answered access the information stored in the views.

Data/Query/Views



- 1 Tom provides detailed profile information to FaceSpace (data), but chooses to limit what is publicly accessible (views).

- 2 Only the public information about Tom becomes data to the advertiser after scraping his profile.

- Rawness
- Immutability
- Eternal Trueness

- Storing raw data is hugely valuable because you rarely know in advance all the questions you want answered.
- Big Data technologies are designed to manage petabytes and exabytes of data.
- We argue that it's better to store the unstructured string, because your semantic normalization algorithm may improve over time (depends upon the algorithm).
- More information doesn't necessarily mean rawer data

Immutability



- *Human-fault tolerance*
- *Simplicity*

Name data		
user id	name	timestamp
1	Alice	2012/03/29 08:12:24
2	Bob	2012/04/12 14:47:51
3	Tom	2012/04/04 18:31:24
4	Charlie	2012/04/09 11:52:30
...	...	...

Age data		
user id	age	timestamp
1	25	2012/03/29 08:12:24
2	36	2012/04/12 14:47:51
3	28	2012/04/04 18:31:24
4	25	2012/04/09 11:52:30
...	...	...

1
Each field of user
information is
kept separately.

Location data		
user id	location	timestamp
1	Atlanta, GA	2012/03/29 08:12:24
2	Chicago, IL	2012/04/12 14:47:51
3	San Francisco, CA	2012/04/04 18:31:24
4	Washington, DC	2012/04/09 11:52:30
...	...	...

2
Each record is
timestamped
when it is stored.

Immutability



- *Human-fault tolerance*
- *Simplicity*

Location data		
user id	location	timestamp
1	Atlanta, GA	2012/03/29 08:12:24
2	Chicago, IL	2012/04/12 14:47:51
3	San Francisco, CA	2012/04/04 18:31:24
4	Washington, DC	2012/04/09 11:52:30
3	Los Angeles, CA	2012/06/17 20:09:48
...	...	...

1 The initial information provided by Tom (user id 3), timestamped when he first joined FaceSpace.

2 When Tom later moves to a new location, you add an additional record timestamped by when you received the new data.

Transient Operational Data



Before image

K1234	abcdef	04/22/2015	750
-------	--------	------------	-----

Update

K1234

04/27/2015

-50

Event (withdrawal)

After image

K1234	abcdef	04/27/2015	700
-------	--------	------------	-----

Table X (10/09)

Key	A	B
001	a	b
002	c	d
003	e	f
004	g	h

Table X (10/10)

Key	A	B
001	a	b
002	r	d
003	e	f
004	y	h
005	m	n

Table X (10/11)

Key	A	B
001	a	b
002	r	d
003	e	t
005	m	n

Periodic Warehouse Data



Table X (10/09)

Key	Date	A	B	Action
001	10/09	a	b	C
002	10/09	c	d	C
003	10/09	e	f	C
004	10/09	g	h	C

Table X (10/10)

Key	Date	A	B	Action
001	10/09	a	b	C
002	10/09	c	d	C
▶ 002	10/10	r	d	U
003	10/09	e	f	C
004	10/09	g	h	C
▶ 004	10/10	y	h	U
▶ 005	10/10	m	n	C

Table X (10/11)

Key	Date	A	B	Action
001	10/09	a	b	C
002	10/09	c	d	C
002	10/10	r	d	U
003	10/09	e	f	C
▶ 003	10/11	e	t	U
004	10/09	g	h	C
004	10/10	y	h	U
▶ 004	10/11	y	h	D
005	10/10	m	n	C

- *Garbage collection*
- *Regulations*



- *Atomic*
 - Facts are atomic because they can't be subdivided further into meaningful components.
 - No redundancy across distinct facts.
- *Timestamped*
 - The timestamps make each fact immutable and eternally true.
- *Identifiability*
 - Ensures each fact is identifiable so that query processing can identify duplicates.

Master dataset will be an ever-growing list of immutable, atomic facts

- General Data Protection Regulation (Reg. UE 2016/679)
- Normativa europea su raccolta, trattamento e conservazione dei dati personali
- Obiettivo: tutelare i diritti delle persone nel trattamento dei dati
- Nei progetti Big Data i dati non sono solo numeri: spesso sono informazioni su persone reali

Dati personali e sensibili



- Dati personali: ogni informazione che identifica o rende identificabile una persona
- Esempi: nome, email, username, indirizzo IP, coordinate GPS
- Dati sensibili: salute, opinioni politiche, religione, etnia, orientamento sessuale
- Richiedono più cautele

Principi chiave del GDPR



- **Liceità, correttezza, trasparenza**

→ Il trattamento dei dati deve avere una base legale (es. consenso, interesse legittimo, obbligo legale), deve essere svolto in modo leale e gli utenti devono sapere chiaramente come i loro dati vengono usati.

Esempio: se raccolgo dati social per analisi accademica, devo dichiararlo e non posso riutilizzarli per fini commerciali senza informare gli utenti.

- **Limitazione della finalità**

→ I dati possono essere utilizzati solo per gli scopi specifici per cui sono stati raccolti, non per altri usi non dichiarati.

Esempio: se raccolgo dati sanitari per una ricerca clinica, non posso riutilizzarli per marketing.

- **Minimizzazione dei dati**

→ Posso raccogliere solo i dati strettamente necessari a raggiungere lo scopo dichiarato, evitando di acquisire informazioni superflue.

Esempio: se analizzo hashtag politici, non è necessario salvare anche l'email o il numero di telefono degli utenti.

Principi chiave del GDPR



- **Accuratezza**

→ I dati devono essere corretti, aggiornati e, se necessario, rettificati o eliminati.

Esempio: se un dataset contiene duplicati o errori, vanno rimossi o segnalati, altrimenti si rischiano analisi distorte.

- **Limitazione della conservazione**

→ I dati non devono essere conservati oltre il tempo necessario allo scopo per cui sono stati raccolti.

Esempio: i log di accesso a un servizio possono essere mantenuti per motivi di sicurezza per alcuni mesi, ma non per anni senza giustificazione.

- **Integrità e riservatezza**

→ I dati devono essere protetti con misure tecniche e organizzative adeguate contro accessi non autorizzati, perdite o divulgazioni indebite.

Esempio: cifratura dei database, gestione degli accessi con ruoli e controlli, backup sicuri.

Implicazioni per i Big Data



- Open Data pubblici → generalmente utilizzabili senza problemi
- Dati da social API → contengono identificativi → attenzione ad anonimizzazione/pseudonimizzazione
- Dati sensibili (sanitari, politici, ecc.) → da evitare o usare solo se già anonimi
- Anche un semplice username è un dato personale!

Anonimizzazione vs Pseudonimizzazione



- Anonimizzazione: irreversibile → non si può risalire all'individuo
- Pseudonimizzazione: sostituzione con codici (es. user_001), ma chi ha la chiave può risalire
- In didattica/research spesso si usano dati pseudonimizzati per analisi senza esporre identità reali

- Il GDPR non è solo 'legge': è responsabilità etica e professionale
- Nei progetti Big Data: privilegiare dataset open/anonimi
- Minimizzare ciò che si raccoglie
- Discutere implicazioni etiche e legali
- Essere data scientist significa anche essere data steward

- Governance: chi gestisce i dati e chi può accedervi
- Data quality: completezza, consistenza, gestione duplicati e missing values
- Fondamentale per trasformare i Big Data in informazioni affidabili

- Controlli di accesso: ruoli e privilegi
- Cifratura: protezione dei dati in transito e a riposo
- Logging e auditing: tracciare chi accede ai dati e quando
- Aspetti cruciali nei cluster distribuiti (MongoDB, HDFS, ecc.)

- Bias nei dati: modelli che riflettono discriminazioni
- Trasparenza: spiegare come vengono prese decisioni basate su Big Data
- Uso improprio: sorveglianza, manipolazione, disinformazione
- Aspetti etici da discutere insieme agli aspetti tecnici

- Non tutti i dataset open sono liberi da vincoli
- Esempi: CC-BY (citare autore), CC0 (libero), ODbL (open database license)
- Leggere sempre la licenza prima di usare i dati
- Implicazioni pratiche per ricerca, tesi, startup

Privacy-preserving Analytics



- Tecniche emergenti per analizzare dati proteggendo la privacy
- Esempi: differential privacy, k-anonymity, federated learning
- Utile come punto di riflessione sulle direzioni future della ricerca



Hadoop: HDFS and beyond

Consistency vs Efficiency

- 1 Data is normalized in the master dataset for compactness and consistency...

Batch layer



Master dataset

- 3 The batch views are continually rebuilt from the master dataset, so all changes are consistent across the batch views.

- 2 ... but is redundantly stored (denormalized) in the batch views for efficient querying.

Serving layer



Batch view



Batch view

Normalized vs Denormalized

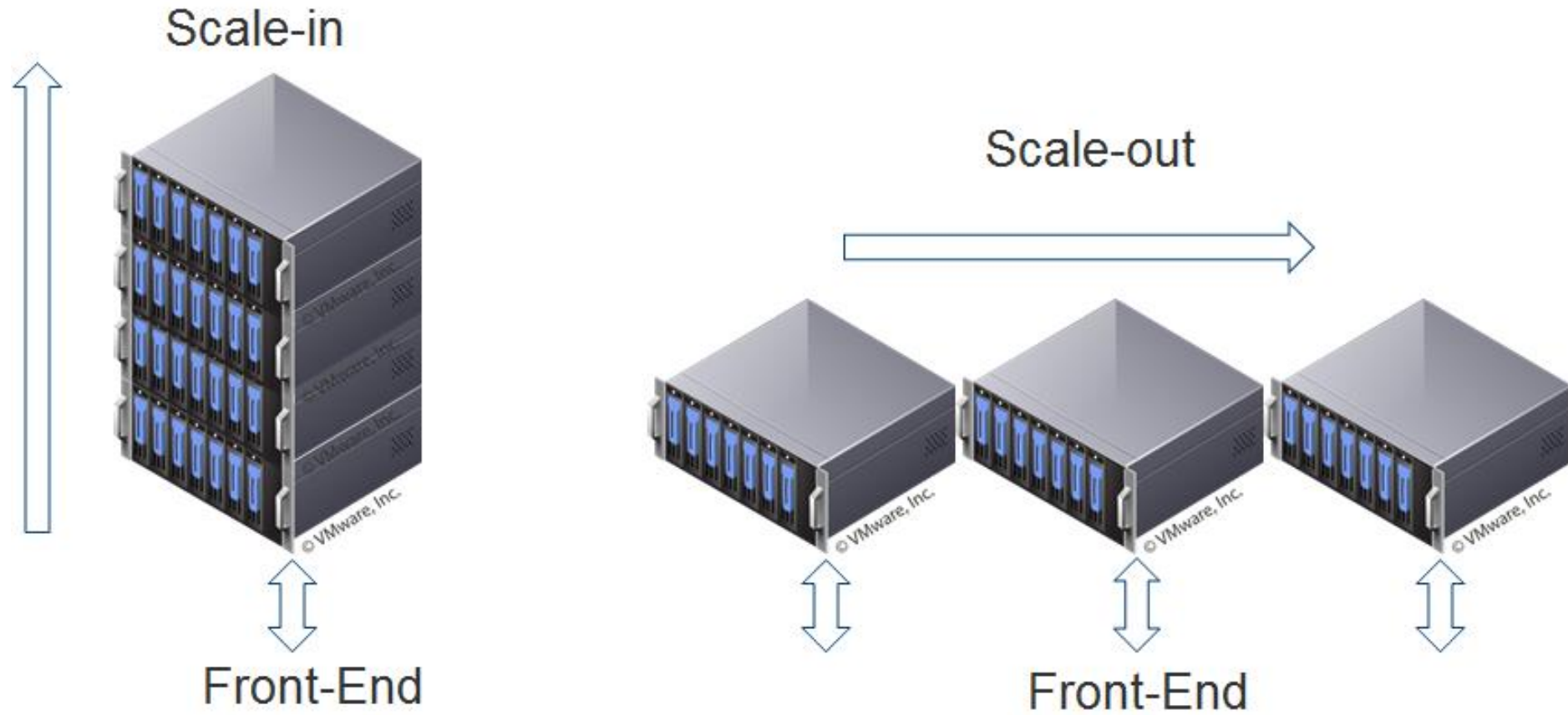
Query: "How many?"

Master Data Storage Requirements

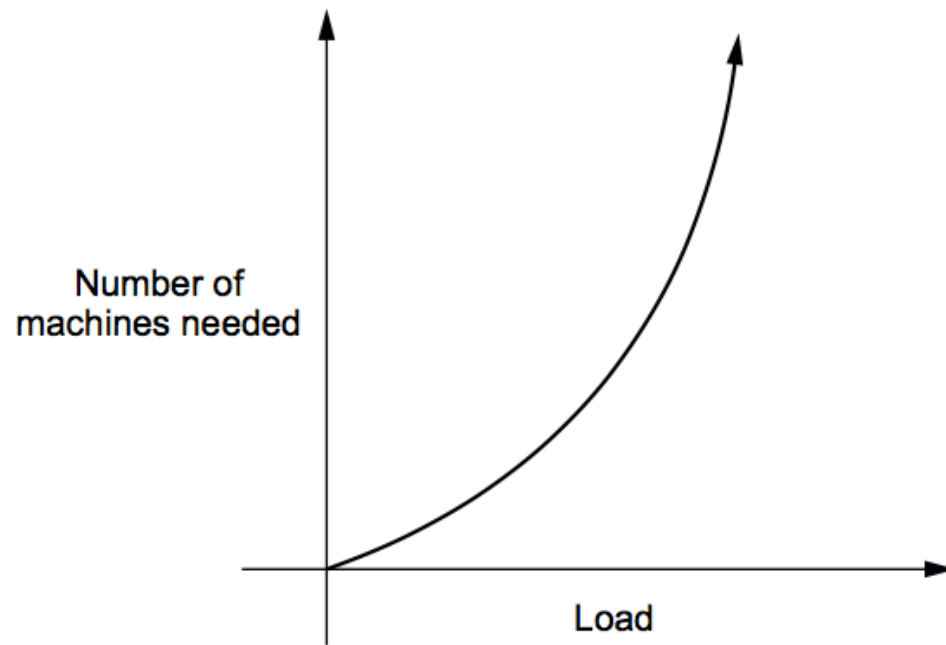


Operation	Requisite	Discussion
Write	Efficient appends of new data	The only write operation is to add new pieces of data, so it must be easy and efficient to append a new set of data objects to the master dataset.
	Scalable storage	The batch layer stores the complete dataset—potentially terabytes or petabytes of data. It must therefore be easy to scale the storage as your dataset grows.
Read	Support for parallel processing	Constructing the batch views requires computing functions on the entire master dataset. The batch storage must consequently support parallel processing to handle large amounts of data in a scalable manner.
Both	Tunable storage and processing costs	Storage costs money. You may choose to compress your data to help minimize your expenses, but decompressing your data during computations can affect performance. The batch layer should give you the flexibility to decide how to store and compress your data to suit your specific needs.
	Enforce-able immutability	It's critical that you're able to enforce the immutability property on your master dataset. Of course, computers by their very nature are mutable, so there will always be a way to mutate the data you're storing. The best you can do is put checks in place to disallow mutable operations. These checks should prevent bugs or other random errors from trampling over existing data.

Scalable Storage



Linearly scalable system



- **Scalability** is the ability of a system to maintain performance under increased load by adding more resources.
- **Linearly scalable system** can maintain performance under increased load by adding resources in proportion to the increased load.

DFS – Distributed File Systems



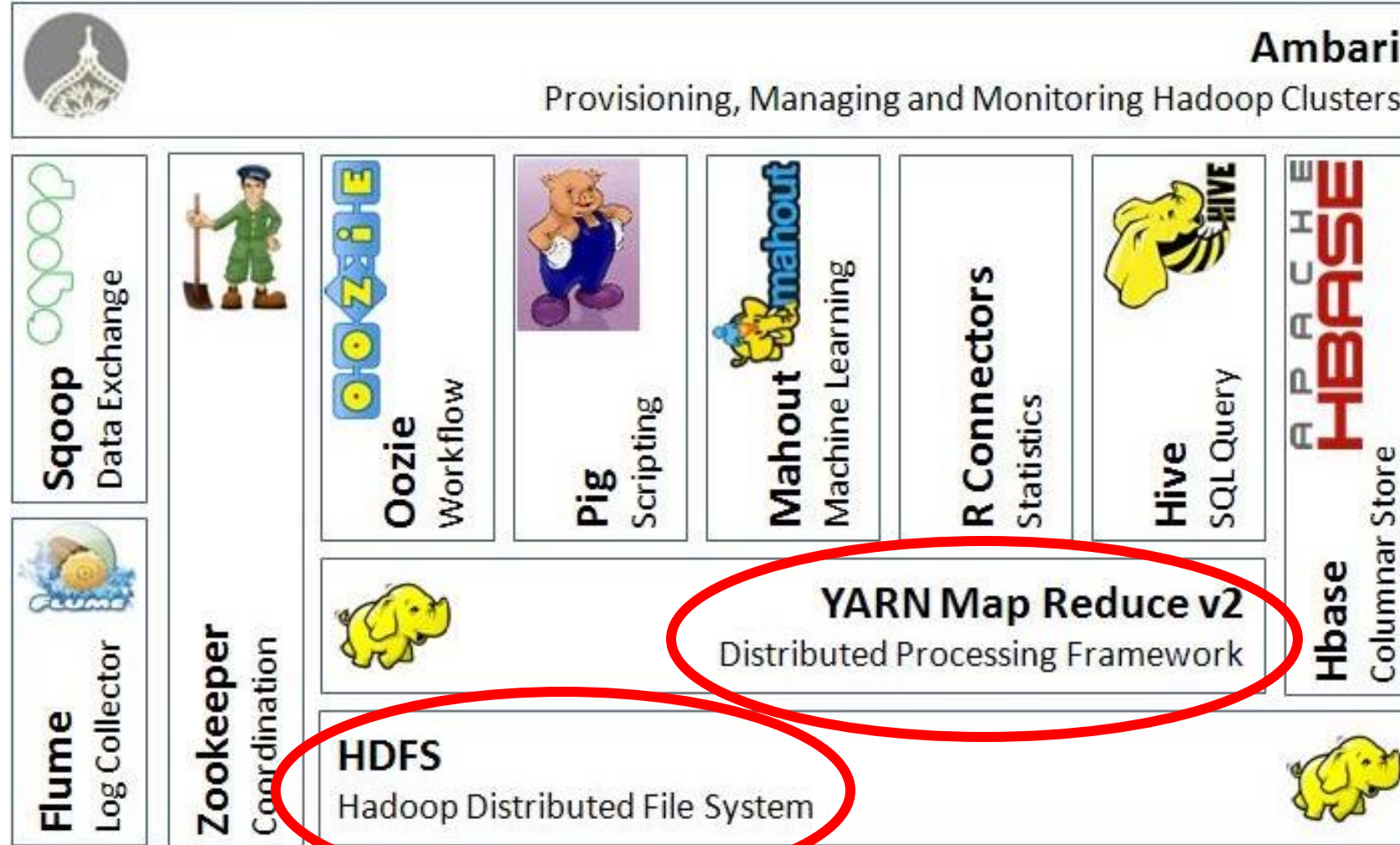
Operation	Requisite	Discussion
Write	Efficient appends of new data	Appending new data is as simple as adding a new file to the folder containing the master dataset.
	Scalable storage	Distributed filesystems evenly distribute the storage across a cluster of machines. You increase storage space and I/O throughput by adding more machines.
Read	Support for parallel processing	Distributed filesystems spread all data across many machines, making it possible to parallelize the processing across many machines. Distributed filesystems typically integrate with computation frameworks like MapReduce to make that processing easy to do (discussed in chapter 6).
Both	Tunable storage and processing costs	Just like regular filesystems, you have full control over how you store your data units within the files. You choose the file format for your data as well as the level of compression. You're free to do individual record compression, block-level compression, or neither.
	Enforceable immutability	Distributed filesystems typically have the same permissions systems you're used to using in regular filesystems. To enforce immutability, you can disable the ability to modify or delete files in the master dataset folder for the user with which your application runs. This redundant check will protect your previously existing data against bugs or other human mistakes.

DFS – Distributed File Systems



- Several distributed file systems used in practice
- The Google File System (GFS), the original of the class
- CloudStore, an open-source DFS originally developed by Kosmix
- Hadoop Distributed File System (HDFS), an open-source DFS used with Hadoop
 - Master node = Namenode
 - Compute node = Datanode
 - Node: both physical and logical entity

Hadoop Ecosystem



- Very Large Distributed File System
 - 10K nodes, 100 million files, 10PB
- Assumes Commodity Hardware
 - Files are replicated to handle hardware failure
 - Detect failures and recover from them
- Optimized for Batch Processing
 - Data locations exposed so that computations can move to where data resides
 - Provides very high aggregate bandwidth

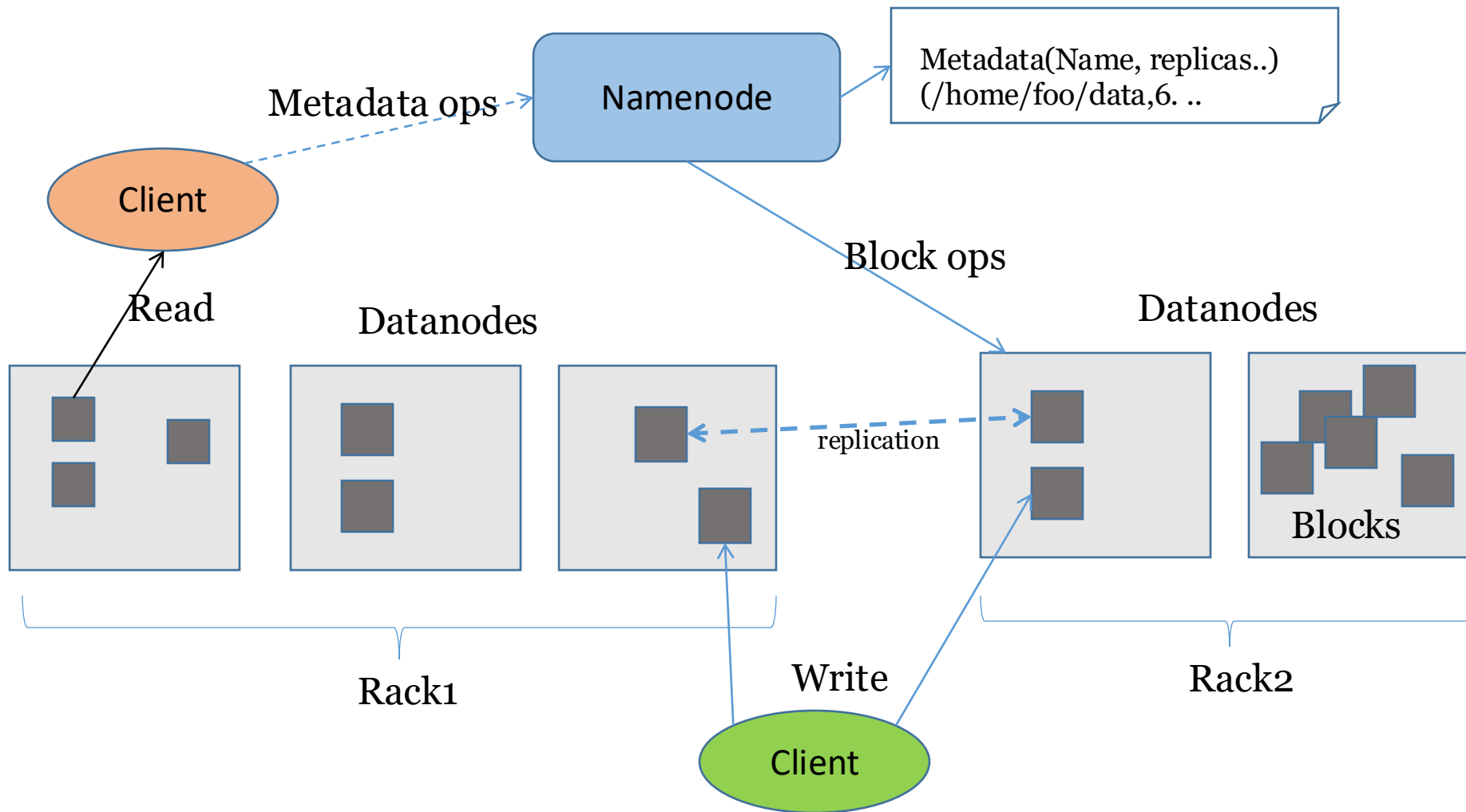


Building blocks of Hadoop



- NameNode
- DataNode
- Secondary NameNode
- JobTracker
- TaskTracker

HDFS Architecture

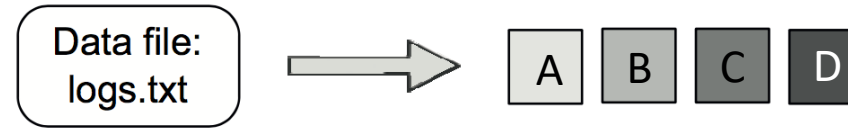


- Files are split into blocks that are spread among many nodes in the cluster.
- Blocks are replicated among many nodes so the data is still available even when machines go down.
- The namenode keeps track of the blocks for each file and where those blocks are stored.

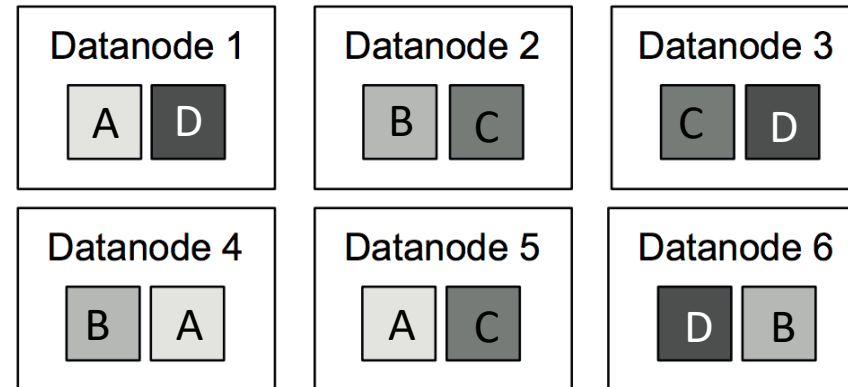
Namenode and Datanodes



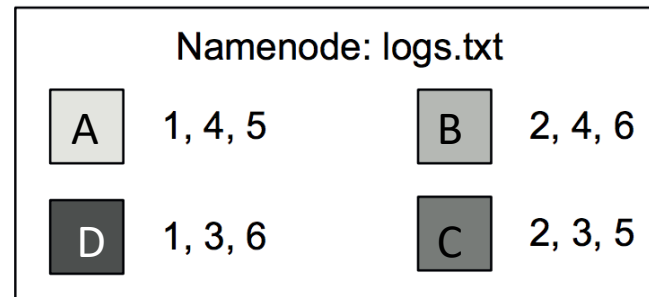
- Master/slave architecture
- HDFS cluster consists of a single **Namenode**, a master server that manages the file system namespace and regulates access to files by clients.
- There are a number of **DataNodes** usually one per node in a cluster.
- The **DataNodes** manage storage attached to the nodes that they run on.
- **HDFS** exposes a file system namespace and allows user data to be stored in files.
- A file is split into one or more blocks and set of blocks are stored in **DataNodes**.
- **DataNodes**: serves read, write requests, performs block creation, deletion, and replication upon instruction from **Namenode**.



- 1 All (typically large) files are broken into blocks, usually 64 to 256 MB.

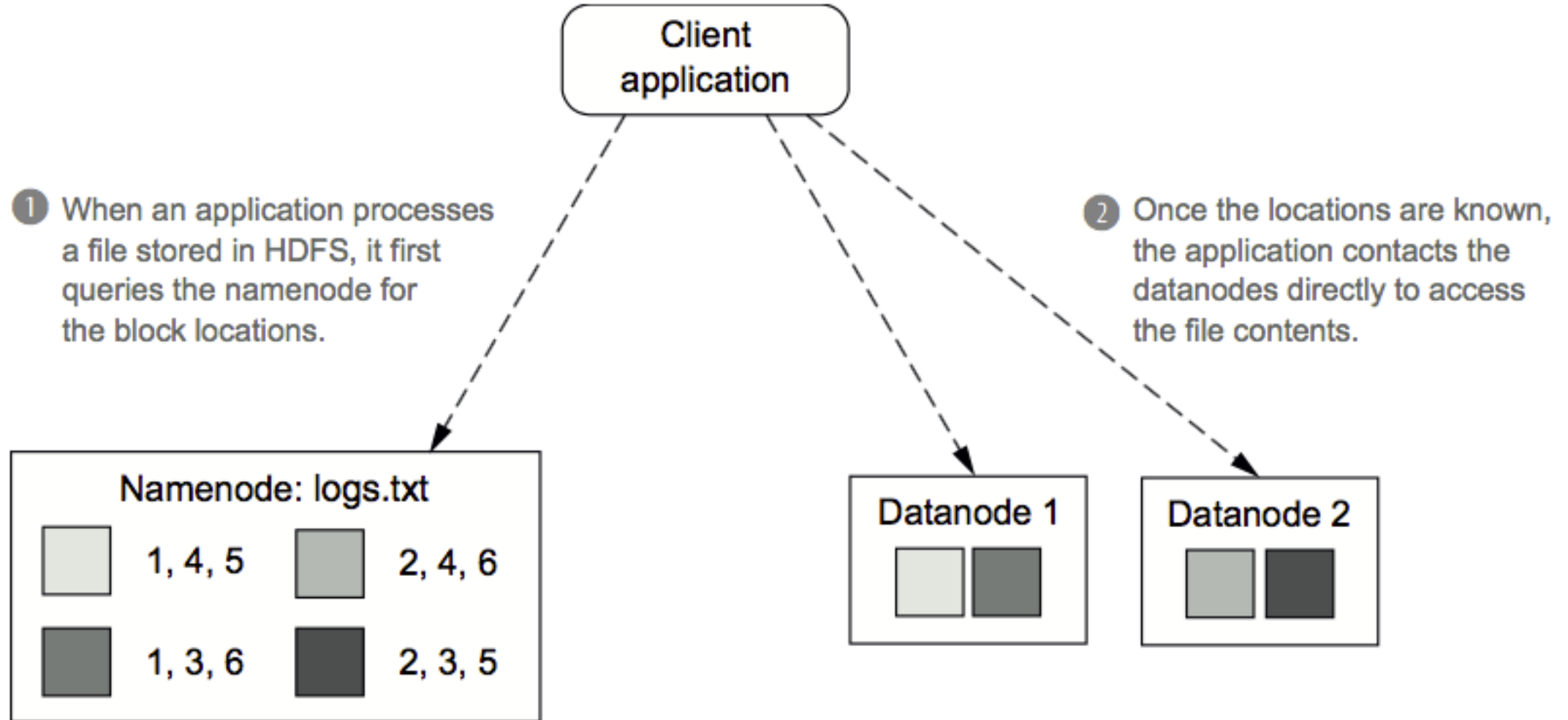


- 2 These blocks are replicated (typically with 3 copies) among the HDFS servers (datanodes).



- 3 The namenode provides a lookup service for clients accessing the data and ensures the blocks are correctly replicated across the cluster.

HDFS



NameNode



- The NameNode is the master of HDFS that directs the slave DataNode daemons to perform the low-level I/O tasks.
- The NameNode is the bookkeeper of HDFS; it keeps track of how your files are broken down into file blocks, which nodes store those blocks, and the overall health of the distributed filesystem.
- NameNode typically doesn't store any user data or perform any computations for a MapReduce program to lower the workload on the machine.
- **It's a single point of failure of your Hadoop cluster**

NameNode Metadata



- Metadata in Memory
 - The entire metadata is in main memory
 - No demand paging of metadata
- Types of metadata
 - List of files
 - List of Blocks for each file
 - List of DataNodes for each block
 - File attributes, e.g. creation time, replication factor
- A Transaction Log
 - Records file creations, file deletions etc

Datanode



- A Datanode stores data in files in its local file system.
- Datanode has no knowledge about HDFS filesystem
- It stores each block of HDFS data in a separate file.
- Datanode does not create all files in the same directory.
- It uses heuristics to determine optimal number of files per directory and creates directories appropriately:
 - Research issue?
- When the filesystem starts up it generates a list of all HDFS blocks and send this report to Namenode: Blockreport.

DataNode failure and heartbeat



- A network partition can cause a subset of Datanodes to lose connectivity with the Namenode.
- Namenode detects this condition by the absence of a Heartbeat message.
- Namenode marks Datanodes without Heartbeat and does not send any IO requests to them.
- Any data registered to the failed Datanode is not available to the HDFS.
- Also the death of a Datanode may cause replication factor of some of the blocks to fall below their specified value.

Secondary NameNode



- The Secondary NameNode (SNN) is an assistant daemon for monitoring the state of the cluster HDFS.
- It typically resides on its own machine as well.
- The SNN differs from the NameNode in that this process doesn't receive or record any real-time changes to HDFS.
- It communicates with the NameNode to take snapshots of the HDFS metadata at intervals defined by the cluster configuration.
- SNN snapshots help minimize the downtime and loss of data.



HDFS

- API (Accessibility)

Operational modes of Hadoop



- ***Local (standalone) mode***

- The standalone mode is the default mode for Hadoop. When you first uncompress the Hadoop source package, it's ignorant of your hardware setup. Hadoop chooses to be conservative and assumes a minimal configuration. Its primary use is for developing and debugging the application logic of a MapReduce program without the additional complexity of interacting with the daemons.

- ***Pseudo-distributed mode***

- The pseudo-distributed mode is running Hadoop in a “cluster of one” with all daemons running on a single machine.

- ***Fully distributed mode***

- After continually emphasizing the benefits of distributed storage and distributed computation, it's time for us to set up a full cluster.
- In core-site.xml and mapred-site.xml we specify the hostname and port of the NameNode and the JobTracker, respectively. In hdfs-site.xml we specify the default replication factor for HDFS, which should only be one because we're running on only one node.

Configuration



Hadoop Configuration in `$HADOOP_HOME/etc/hadoop`:

- **core-site.xml**: common settings for HDFS, MapReduce, and YARN
- **hdfs-site.xml**: configuration settings for HDFS daemons (i.e., namenode, secondary namenode, and datanodes)
- **mapred-site.xml**: configuration settings for MapReduce (e.g., job history server)
- **yarn-site.xml**: configuration settings for YARN daemons (e.g., resource manager, node managers)

By default, Hadoop runs in a non-distributed mode, as a single Java process. We will configure Hadoop to execute in a pseudo-distributed mode

More on the Hadoop configuration: <https://hadoop.apache.org/docs/current/>

Configuration



core-site.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
|
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

hdfs-site.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
|
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
</configuration>
```

Configuration



mapred-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

yarn-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Application Programming Interface



- HDFS provides [Java API](#) for application to use.
- [Python](#) access is also used in many applications.
- A C language wrapper for Java API is also available.
- A HTTP browser can be used to browse the files of a HDFS instance.

FS Shell, Admin and Browser Interface



- HDFS organizes its data in files and directories.
- It provides a command line interface called the FS shell that lets the user interact with data in the HDFS.
- The syntax of the commands is similar to bash and csh.
- Example: to create a directory /foodir

```
/bin/hadoop dfs -mkdir /foodir
```
- There is also DFSAdmin interface available
- Browser interface is also available to view the namespace.

`./bin/hdfs [-cmd <args>]`

- *cmd* may be:

- ls
- format
- mkdir
- rm (rmdir for directory)
- get
- put
- cat

Reading and writing to HDFS programmatically

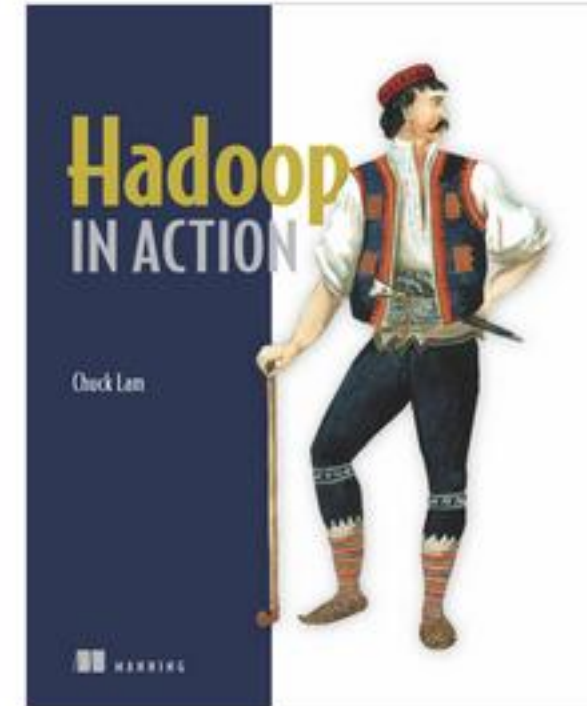


```
public static void main(String[] args) throws IOException {  
    Configuration conf = new Configuration();  
    FileSystem hdfs = FileSystem.get(conf);  
    FileSystem local = FileSystem.getLocal(conf);  
  
    Path inputDir = new Path(args[0]);  
    Path hdfsFile = new Path(args[1]);  
  
    try {  
        FileStatus[] inputFiles = local.listStatus(inputDir);  
        FSDataOutputStream out = hdfs.create(hdfsFile);  
  
        for (int i=0; i<inputFiles.length; i++) {  
            System.out.println(inputFiles[i].getPath().getName());  
            FSDataInputStream in =  
➡ local.open(inputFiles[i].getPath());  
            byte buffer[] = new byte[256];  
            int bytesRead = 0;  
            while( (bytesRead = in.read(buffer)) > 0) {  
                out.write(buffer, 0, bytesRead);  
            }  
            in.close();  
        }  
        out.close();  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

1

3

4



Chapter 3rd



Hands on HDFS and DOCKER

HDFS and Docker



```
$ docker pull effeerre/hadoop
```

- create a small network named **hadoop_network** with one namenode (**master**) and 3 datanodes (**slave**)

```
$ docker network create --driver bridge hadoop_network
```

```
$ docker run -t -i -p 9864:9864 -d --network=hadoop_network  
--name=slave1 effeerre/hadoop
```

```
$ docker run -t -i -p 9863:9864 -d --network=hadoop_network  
--name=slave2 effeerre/hadoop
```

```
$ docker run -t -i -p 9862:9864 -d --network=hadoop_network  
--name=slave3 effeerre/hadoop
```

```
$ docker run -t -i -p 9870:9870 --network=hadoop_network  
--name=master effeerre/hadoop
```

How to remove the containers

- stop and delete the namenode and datanodes

```
$ docker kill slave1 slave2 slave3  
$ docker rm master slave1 slave2 slave3
```

- remove the network

```
$ docker network rm hadoop_network
```

Hands on HDFS



At the first execution, the HDFS needs to be initialized

```
$ hdfs namenode -format
```

- this operation **erases the content of the HDFS**
- it should be executed only during the initialization phase

Start HDFS:

```
$ $HADOOP_HOME/sbin/start-dfs.sh
```

Stop HDFS:

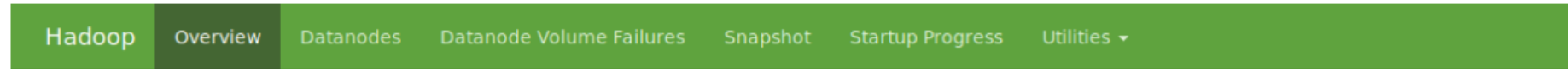
```
$ $HADOOP_HOME/sbin/stop-dfs.sh
```

Hands on HDFS



When the HDFS is started, you can check its WebUI:

- <http://localhost:9870/>



Overview 'master:54310' (active)

Started:	Mon Feb 25 22:25:04 +0100 2019
Version:	3.1.2, r1019dde65bcf12e05ef48ac71e84550d589e5d9a
Compiled:	Tue Jan 29 02:39:00 +0100 2019 by sunilg from branch-3.1.2
Cluster ID:	CID-389ffd11-3b1a-4051-9b8e-f85812a659df
Block Pool ID:	BP-358197840-172.18.0.5-1551129887862

```
$ $HADOOP_HOME/sbin/stop-dfs.sh
```

Obtain basic filesystem information and statistics:

```
$ hdfs dfsadmin -report
```

Hands on HDFS



ls: for a file ls returns stat on the file; for a directory it returns list of its direct children

```
$ hdfs dfs -ls [-d] [-h] [-R] <args>
```

- d: Directories are listed as plain files
- h: Format file sizes in a human-readable fashion
- R: Recursively list subdirectories encountered

mkdir: takes path uri's as argument and creates directories

```
$ hdfs dfs -mkdir [-p] <paths>
```

- p: creates parent directories along the path.

Hands on HDFS



mv: moves files from source to destination. This command allows multiple sources in which case the destination needs to be a directory. Moving files across file systems is not permitted

```
$ hdfs dfs -mv URI [URI ...] <dest>
```

put: copy single src, or multiple srcs from local file system to the destination file system

```
$ hdfs dfs -put <localsrc> ... <dst>
```

Also reads input from stdin and writes to destination file system

```
$ hdfs dfs -put - <dst>
```

Hands on HDFS



append: append single or multiple files from local file system to the destination file system

```
$ hdfs dfs -appendToFile <localsrc> ... <dst>
```

get: copy files to the local file system; files that fail the CRC check may be copied with the `-ignorecrc` option

```
$ hdfs dfs -get [-ignorecrc] [-crc] <src> <localdst>
```

cat: copies source paths to stdout

```
$ hdfs dfs -cat URI [URI ...]
```

rm: Delete files specified as args

```
$ hdfs dfs -rm [-f] [-r | -R] [-skipTrash] URI [URI ...]
```

- f: does not display a diagnostic message (modify the exit status to reflect an error if the file does not exist)
- R (or -r): deletes the directory and any content under it recursively
- skipTrash: bypasses trash, if enabled

cp: copy files from source to destination. This command allows multiple sources as well in which case the destination must be a directory

```
$ hdfs dfs -cp [-f] [-p | -p[topax]] URI [URI ...] <dest>
```

- f: overwrites the destination if it already exists.
- p: preserves file attributes [topx] (timestamps, ownership, permission, ACL, XAttr). If -p is specified with no arg, then preserves timestamps, ownership, permission.

stat: Print statistics about the file/directory at <path> in the specified format

```
$ hadoop fs -stat [format] <path> ...
```

Format accepts

%b Size of file in bytes

%F Will return "file", "directory", or "symlink" depending on the type of inode

%g Group name

%n Filename

%o HDFS Block size in bytes (128MB by default)

%r Replication factor

%u Username of owner

%y Formatted mtime of inode

%Y UNIX Epoch mtime of inode

setrep: change the replication factor of a file. If path is a directory then the command recursively changes the replication factor of all files under the directory tree rooted at path.

```
$ hdfs dfs -setrep [-w] <numReplicas> <path>
```

-w: requests that the command wait for the replication to complete;
this can potentially take a very long time

Example



```
$ echo "File content" >> file
$ hdfs dfs -put file /file
$ hdfs dfs -ls /
$ hdfs dfs -mv /file /democontent
$ hdfs dfs -cat /democontent
$ hdfs dfs -appendToFile file /democontent
$ hdfs dfs -cat /democontent
$ hdfs dfs -mkdir /folder01
$ hdfs dfs -cp /democontent /folder01/text
$ hdfs dfs -ls /folder01
$ hdfs dfs -rm /democontent
$ hdfs dfs -get /folder01/text textfromhdfs
$ cat textfromhdfs
$ hdfs dfs -rm -r /folder01
```

Exercise



- Add some text files to hdfs
- Increment the replication factor
- Don't forget to give an outlook to the configuration "hdfs-site.xml»
- Later, when you are familiar with HDFS API, let's try to do your best for starting HDFS in distributed mode and try to upload some files there considering for instance following dataset sources:
 - <https://www.springboard.com/blog/free-public-data-sets-data-science-project/>

